

OCR A Level Computer Science H446-03

Programming Project

PITWALL

Motorsport Race Strategy Simulation and Optimisation System

Candidate: Mithil Katkoria

Candidate number: 0460

Centre number: 12709

NEA final master document

PITWALL compares race strategies using an explainable mathematical model. This report presents the problem, design, implementation, recorded tests and evaluation, followed by supporting appendices.

The current evidence supports computational correctness within the stated model. Stakeholder investigation and human usability results remain to be collected. Precise placeholders identify these gaps.

Contents

Abbreviations	5
1 Analysis of the problem	6
1.1 Introduction	6
1.2 Description of the problem	6
1.3 Why computation is appropriate	6
1.4 Stakeholders.....	6
1.5 Research into existing solutions.....	6
1.6 Proposed computational solution.....	9
1.7 Essential features	9
1.8 Scope limitations.....	10
1.9 Hardware and software requirements	10
1.10 Success criteria.....	10
2 Design of the solution	13
2.1 Problem decomposition.....	13
2.2 Overall system structure	13
2.3 Class design	14
2.4 Data structures.....	15
2.5 Important variables.....	16
2.6 Algorithm design	16
2.7 Validation design.....	21
2.8 Usability design	22
2.9 Current interface design	23
2.10 Iterative test plan.....	23
2.11 Post-development test plan.....	24
3 Development and testing.....	25
3.1 Core models	25
3.2 Tyre degradation	25
3.3 Lap and fuel calculation	26
3.4 Pit stops and dry race.....	27
3.5 Strategy comparison	28
3.6 First desktop milestone.....	28
3.7 Event queue	30
3.8 Weather effects	31
3.9 Safety car.....	31
3.10 Seeded variation	32
3.11 Monte Carlo and statistics	33
3.12 Bounded optimisation.....	33

3.13 JSON persistence.....	34
3.14 Advanced GUI and charts.....	35
3.15 Robustness and verification.....	36
3.16 Genuine failures and remedial action.....	37
3.17 Recorded test progression.....	38
4 Post-development testing.....	40
4.1 Functional testing.....	40
4.2 Robustness testing.....	40
4.3 Performance testing.....	41
4.4 Manual verification.....	42
4.5 Usability testing.....	42
4.6 Final quality verification.....	43
5 Evaluation.....	45
5.1 Overall evaluation.....	45
5.2 Evaluation against success criteria.....	45
5.3 Functional correctness.....	46
5.4 Evaluation of algorithms.....	47
5.5 Performance.....	47
5.6 Robustness.....	47
5.7 Usability.....	47
5.8 Stakeholder evaluation.....	47
5.9 Maintenance.....	48
5.10 Limitations.....	48
5.11 Future improvements.....	49
5.12 Final conclusion.....	49
6 Bibliography.....	50
7 Appendices.....	51
Appendix A Source and class relationships.....	51
Appendix B Automated test inventory.....	52
Appendix C Execution and failure evidence.....	54
Appendix D Full benchmark values.....	55
Appendix E Research materials.....	57
Appendix F Human evidence packs.....	57
Appendix G Git development summary.....	62
Appendix H Evidence catalogue and completeness audit.....	62
Appendix I JSON contract and example.....	64
Appendix J Additional application screenshots.....	65
Appendix K Development declaration.....	68

List of figures

Figure 1: Implemented module architecture

Figure 2: Current strategy comparison interface

Figure 3: First deterministic GUI milestone

Figure 4: Completed bounded-search result

Figure 5: Core class relationships

Figure 6: Actual comparison metrics

Figure 7: Actual lap records

Figure 8: Conditions and experiment settings

Figure 9: Actual paired Monte Carlo summary

Figure 10: Actual Monte Carlo histogram

List of tables

Table 1: Proposed stakeholder involvement

Table 2: Existing solution comparison

Table 3: Existing solution comparison

Table 4: Existing solution comparison

Table 5: Requirements from the project brief

Table 6: Development and end-user environment

Table 7: First milestone success criteria

Table 8: Advanced success criteria

Table 9: Classes in models

Table 10: Classes in events

Table 11: Classes in conditions

Table 12: Classes in monte_carlo

Table 13: Classes in optimiser

Table 14: Classes in persistence

Table 15: Structures and alternatives

Table 16: Variables and units

Table 17: Algorithm example or test data

Table 18: Core data responsibilities

Table 19: Typical boundary and erroneous tests

Table 20: All preserved execution runs

Table 21: Functional verification

Table 22: Recorded invalid and boundary cases

Table 23: Raw measured seconds and summary

Table 24: Five-lap independent expected calculation

Table 25: Unconducted usability task sheet

Table 26: Comparison latency summary

Table 27: Raw comparison repeats in seconds

Table 28: Success criterion judgements

Table 29: Algorithm strengths and trade-offs

Table 30: Maintenance issues and technical changes

Table 31: Limitations and improvements

Table 32: Final collected cases

Table 33: Blank human evidence form

Table 34: Blank human evidence form

Table 35: Blank human evidence form

Table 36: Candidate understanding topics

Table 37: Genuine commits in chronological order

Table 38: Criterion completeness and remaining work

Table 39: Factual assistance record

Abbreviations

GUI: graphical user interface. JSON: JavaScript Object Notation. RNG: random number generator. SC: safety car in model settings; SC01 to SC14: success criteria. FR: functional requirement. NEA: non-exam assessment. UTC: coordinated universal time.

1 Analysis of the problem

1.1 Introduction

PITWALL is a desktop race-strategy simulator. A strategy identifies the starting tyre and the lap after which each pit stop occurs. The program calculates the consequence of those decisions over the full race and allows alternatives to be compared under the same assumptions.

1.2 Description of the problem

A fresh Soft tyre can be quicker than a Hard tyre, but its pace can deteriorate more rapidly. Remaining on an old set saves pit time immediately but can lose time on every later lap. Race length changes whether a pit stop has enough time to pay back its cost. Fuel and driver pace also affect the total. Weather and safety-car periods add changing conditions, so a plan that is best in a dry race may be less effective when the timing of an event changes.

Manual calculation is practical for a five-lap example, but becomes repetitive across many stop laps and tyre sequences. It is easy to count a pit loss twice or reset tyre age on the wrong lap. A computer can apply one defined rule consistently and retain the components for inspection.

1.3 Why computation is appropriate

Abstraction removes wheel-to-wheel physics and represents a single car with a few time penalties. This keeps the model understandable but limits real-world prediction. Decomposition separates input validation, lap calculation, event state, comparison and presentation. Selection determines whether a stop or event applies. Iteration repeats the lap model and then repeats whole races for candidate strategies or uncertainty trials. Visualisation exposes tyre trends and pit spikes that are difficult to see in a total alone. These techniques solve specific parts of the problem rather than being included only as terminology.

1.4 Stakeholders

Appropriate stakeholder types include motorsport enthusiasts learning strategy trade-offs, sim-racing users comparing plans, and Computer Science students or teachers reviewing the model. These are proposed categories, not invented interview participants. No named participant or response is recorded.

Table 1: Proposed stakeholder involvement

Type	Use and information needed	Response
Motorsport enthusiast	Interpret pit timing and compound trade-offs	[GENUINE STAKEHOLDER RESPONSE REQUIRED]
Sim-racing user	Assess which simplifications affect usefulness	[GENUINE STAKEHOLDER RESPONSE REQUIRED]
Student or teacher	Explain algorithms and check clarity	[GENUINE STAKEHOLDER RESPONSE REQUIRED]

Prepared questions ask which decisions users want to explore, which inputs they understand, what output helps comparison, how errors should appear, which simplifications could mislead them, and whether a pit-lap spike is clear. Appendix F provides a ready-to-use record and task sheet.

1.5 Research into existing solutions

Source review recorded on 2026-09-08 after the implemented prototype. This timing is explicit: these sources did not cause the initial design, which came from the user's brief. Findings below inform maintenance and the research-led review exercises added now. No software was downloaded or personally trialled for this review.

Weaknesses refer to suitability for PITWALL's educational purpose and are identified as judgements, not measured defects.

R01: TUM race-simulation

The authors describe a lap-based simulator with fuel-mass reduction, degradation, participant interactions and Monte Carlo. Their separate basic simulator assumes a free track and finds minimum race time; the broader system includes neural-network strategy approaches. TUM repository and documentation (<https://github.com/TUMFTM/race-simulation>).

Table 2: Existing solution comparison

Feature	Strength	Weakness for this project (judgement)	Decision	PITWALL impact
Basic/full separation	Makes assumptions distinguishable	Full interaction model exceeds initial scope	ADAPT	Keep single-car limitation visible; compare baseline and search under identical assumptions
Lap discretisation	Supports repeated scenario evaluation	Cannot resolve within-lap changes	LEARN	Add review exercise identifying a mid-lap event that PITWALL cannot represent
Monte Carlo	Studies probabilistic effects	Results depend on distributions	ADAPT	Retain seeds and test probability sensitivity rather than claim calibrated confidence
Neural strategy engineer	Automates richer decisions	Harder to defend at A-level	REJECT	Retain bounded enumeration and explain candidate count

No source code or parameter set was copied. The authors' warning about exact race reproduction reinforces the need for PITWALL's conditional claims.

R02: FastF1

The maintainer describes timing/telemetry/session access, extended Pandas structures, Matplotlib integration and request caching. This is a data-analysis library rather than PITWALL's counterfactual strategy simulator. FastF1 maintainer README (<https://github.com/theOehrly/Fast-F1>).

Table 3: Existing solution comparison

Feature	Strength	Weakness for this project (judgement)	Decision	PITWALL impact
Structured timing data	Supports inspecting individual laps	Observed laps contain confounding race effects	LEARN	Add calibration exercise separating observations from model inputs
Matplotlib integration	Connects numeric data and plots	Plot alone cannot establish model correctness	ADOPT principle	Keep independent arithmetic tests alongside graphs
API caching	Reduces repeated retrieval	Introduces external data/cache concerns	REJECT dependency now	Keep scenarios offline and portable
Pandas extensions	Rich analysis tools	Extra abstraction for a small typed model	ADAPT idea	Retain explicit LapResult records instead of adding DataFrames

Future calibration would need source/date/provenance and rules for excluding pit, traffic and weather laps. FastF1 is not evidence that current fictional parameters are accurate. No live feed is required or integrated.

R03: F1 Manager series

Frontier's 2023 Belgian circuit guide discusses mistimed dry/intermediate changes and a one-stop versus extra-stop decision involving traffic. This illustrates a strategy-learning scenario; game guidance is not validated engineering telemetry. Publisher circuit guide (<https://www.f1manager.com/nl-NL/2023/grand-prix/belgian-grand-prix>).

The 2024 feature article describes mechanical failures, more aggressive battles and multiple viewing angles. These are publisher descriptions, not independent realism findings. Publisher feature article (<https://www.f1manager.com/features/new/levensechte-f1-races>).

Table 4: Existing solution comparison

Feature	Strength	Weakness for this project (judgement)	Decision	PITWALL impact
Scenario-based decisions	Gives users a concrete trade-off	Outcome mixes traffic and tyre effects	ADAPT	Add controlled comparison tasks in review materials
Wet tyre timing	Demonstrates transition risk	PITWALL lacks wet compounds	LEARN	Mark wet-tyre choice explicitly unimplemented; propose suitability matrix
Failures and battles	Rich race context	Too broad for explainable first model	REJECT	Maintain single-car abstraction
Race presentation	Helps users follow events	Cinematic view does not reveal formulas	ADAPT	Prioritise component tables and actual graphs

Research-led changes made at this review

Added docs/testing/research-review-exercises.md: inspect a mid-lap event limitation; compare one/two-stop cases; examine probability sensitivity; distinguish telemetry calibration from illustrative input; explicitly identify missing wet tyres. These are new review tasks, not retrospective claims of stakeholder requests or test results.

The initial guessed TUM URL returned 404 and the FastF1 docs endpoint returned 403. The canonical repositories above were then read successfully. Some publisher pages required search retrieval. The accessible source contents support this review; no unavailable page is cited as read.

Post-prototype review and next decisions

The problem is to compare complete race plans, not merely select the fastest fresh tyre. A faster compound may lose its advantage through wear or an additional pit stop. PITWALL keeps a common race configuration, evaluates each racing lap and ranks accumulated times so a user can inspect that trade-off.

Computation is useful because the same arithmetic must be applied consistently across many lap and stop combinations. Decomposition separates input validation, lap components, state transitions and result comparison. A priority queue preserves chronological events and deterministic ties. Bounded enumeration gives a checkable best plan inside an explicit search space. Seeded trials help explore variation but cannot calibrate the model by repetition alone.

The three existing-system reviews were recorded after the prototype. Their value is in reviewing scope: TUM's richer simulator highlights PITWALL's single-car simplification; FastF1 highlights the need for provenance before using real data; F1 Manager suggests clearer presentation but is not evidence for physical parameters. These findings support review exercises and possible future changes, not a rewritten claim that they created the original design. Genuine participant priorities are still required before choosing a new feature.

1.6 Proposed computational solution

The solution builds from validated race and tyre inputs to a deterministic engine. Comparison runs each plan using the same configuration. Conditions add scheduled weather and safety-car events. A local seeded generator controls lap variation. Monte Carlo repeats a paired comparison and records trial seeds; bounded optimisation enumerates legal plans and ranks their totals. JSON preserves scenarios, and the GUI displays results and allows long runs to be cancelled. These functions are implemented, but their suitability for actual users still needs genuine feedback.

1.7 Essential features

Table 5: Requirements from the project brief

ID	Requirement	Source	Priority	Justification	Acceptance
FR01	Configure race length and base pace	Brief	Must	Compare race scenarios	SC01
FR02	Configure Soft, Medium, Hard pace and degradation	Brief	Must	Model compound trade-offs	SC02
FR03	Create strategies with ordered pit stops	Brief	Must	Explore alternative stints	SC03
FR04	Reject invalid values with explanations	Brief	Must	Prevent misleading results	SC04
FR05	Simulate a deterministic dry race	Brief	Must	Establish correctness before uncertainty	SC05
FR06	Show per-lap and total results	Brief	Must	Make calculation inspectable	SC06
FR07	Compare at least two strategies under identical conditions	Brief	Must	Support strategy decisions	SC07
FR08	Plot actual simulated lap times in a desktop GUI	Brief	Must	Expose changes over race distance	SC08
FR09	Save and load a scenario as JSON	Brief	Later	Repeat comparisons	Separate iteration
FR10	Process weather and safety-car events chronologically	Brief	Later	Explore changing conditions	Separate iteration
FR11	Add seed-reproducible randomness and Monte Carlo statistics	Brief	Later	Quantify model uncertainty	Separate iteration
FR12	Search constrained legal strategies and compare baselines	Brief	Later	Evaluate explainable algorithms	Separate iteration

The source of FR01 to FR12 is the supplied project brief. The priorities above describe the recorded development order, not missing implementation: the later features were subsequently built. No requirement is attributed to an unconsulted stakeholder.

1.8 Scope limitations

The initial scope prioritised dry deterministic calculation. The implemented extension still uses only dry compounds, linear degradation and a single-car model. Weather adds equal penalties to all available tyres; safety car changes lap delay and relative pit loss without field bunching. Parameters are illustrative, not proprietary telemetry. Bounded search deliberately excludes off-grid and more-than-two-stop plans. These limits are evaluated in Section 5.10.

1.9 Hardware and software requirements

Table 6: Development and end-user environment

Area	Recorded or required environment	Justification
Developer hardware	Snapdragon(R) X 10-core X1P64100 @ 3.40 GHz; 16756006912 bytes RAM	Actual benchmark machine, not a minimum specification
Developer software	Windows 11; Python 3.14 AMD64; PySide6; Matplotlib; pytest; Git	Desktop UI, graphing, automated checks and genuine history
End user	Tested Python environment and a readable desktop display	Source distribution currently requires dependencies; minimum hardware unmeasured
Installation	Short virtual-environment path on Windows	Avoids recorded PySide6 path-length installation problem

Python expresses the calculations without a compiled build step. Dataclasses and enums are standard-library structures. Qt supplies desktop controls and a worker thread; Matplotlib renders result graphs [R06, R07]. JSON supports a small offline scenario without database administration. The development machine uses an AMD64 Python process on Snapdragon hardware, so the timings must not be described as native ARM measurements.

1.10 Success criteria

Table 7: First milestone success criteria

ID	Criterion and measurement	Link	Justification
SC01	Accept integer race lengths 1..500 and positive base lap time; reject 0, negative, fractional and oversized lengths	FR01	Finite, useful interactive scope
SC02	All three compounds have configurable non-negative pace penalties and linear degradation; age 3 at 0.2 s/lap gives 0.6 s	FR02	Explainable arithmetic
SC03	Ordered distinct stops after laps 1..N-1 change compound on the following lap and add pit loss exactly once	FR03	Unambiguous lap indexing

ID	Criterion and measurement	Link	Justification
SC04	Automated typical, boundary and erroneous cases raise descriptive errors; GUI shows an error for malformed stops	FR04	Actionable validation
SC05	Two runs of identical inputs return equal results; five-lap manual oracle matches each lap within 1e-9 seconds	FR05	Reproducibility and independent oracle
SC06	Exactly N lap rows; total equals sum of lap times; tyre age resets after a stop	FR06	Complete inspectable output
SC07	At least two strategies produce ranked total, average, fastest, slowest, pit loss, stop count and stint lengths	FR07	Useful direct comparison
SC08	Real Qt application opens and renders a graph with N points per compared strategy; manual usability remains separately pending	FR08	Viewable genuine output

These criteria consolidate the measurable conditions recorded before implementation in I08..I17 designs. Stakeholder approval is pending. They do not replace the earlier first-milestone criteria.

Table 8: Advanced success criteria

ID	Requirement	Measurable criterion	Justification	Planned evidence
SC09	FR09	Exact scenario round-trip with seed/events/stop service; reject wrong schema, malformed data and collection types	Repeatable portable inputs	Persistence and final robustness tests
SC10	FR10	Events chronological; ties input-ordered; wet increments30s and SC manual result519.6s in documented five-lap examples	Verify state effects independently	Event and condition tests
SC11	FR11	Same seed reproduces full trials; summary oracle [1,2,3] correct;100 actual paired trials; count0/1001 rejected	Reproducible bounded uncertainty	Monte Carlo tests and benchmark
SC12	FR12	N3/max1 space contains 21 unique plans; tiny winner182.2s; independent4-lap score distribution matches; report complete candidate count	Verify search coverage and scoring	Optimiser/final tests and exported search

ID	Requirement	Measurable criterion	Justification	Planned evidence
SC13	FR11/FR12	Recorded machine default search <10s and 1000 paired 50-lap trials <30s (three repeats)	Usable bounded interactive workloads	Real BENCH-01
SC14	FR09..FR12	Background task returns results; old-input results discarded; cancellation callback honoured; GUI data round-trip exact	Protect responsiveness and result provenance	GUI integration and cancellation tests

The first 13 criteria are computational/visual checks. SC14 needs additional genuine manual responsiveness and cancellation observations before usability is judged fully met. Working widgets are not automatically usable for the intended audience.

2 Design of the solution

Original or recorded design

The I01-I17 design files and actual Git history are retained. Their existence alone does not prove every paragraph preceded every edit. Run metadata supplies source hashes at execution time; seven same-day commits provide coarse checkpoints. The report distinguishes the documented combined stages and actual order.

Post-implementation architecture documentation

Implemented-structure diagrams and the master report class inventory were derived from current source. They explain the completed program and are not original wireframes. Existing-system research is a post-prototype review, with its later timing retained.

New prospective design

No feedback-driven software iteration is claimed. When a genuine response identifies a need, record its alias and evidence reference, update the requirement, write design and test expectations before coding, implement one justified change, run tests, retain any actual failures and request a real follow-up review. Candidate reflection must use the real dates. The final computational verification plan is separate from such an iteration.

2.1 Problem decomposition

The problem is decomposed into: describing valid conditions; predicting one lap; tracking tyre age and accumulated time; applying decisions and changing conditions; comparing complete outcomes; exploring uncertain outcomes; searching a finite decision space; preserving inputs; and making results readable. Validation affects every part because invalid inputs can otherwise produce convincing but meaningless outputs.

2.2 Overall system structure

The actual dependency direction runs from GUI to validated models and analysis functions. Computational modules do not import Qt. There is no separate controller module: AdvancedWindow collects an immutable snapshot and calls the engine or worker. Figure 1 shows these existing modules.

Figure 1 shows how input, calculation and presentation are separated. This diagram was prepared for this report from current source, not presented as an original wireframe.

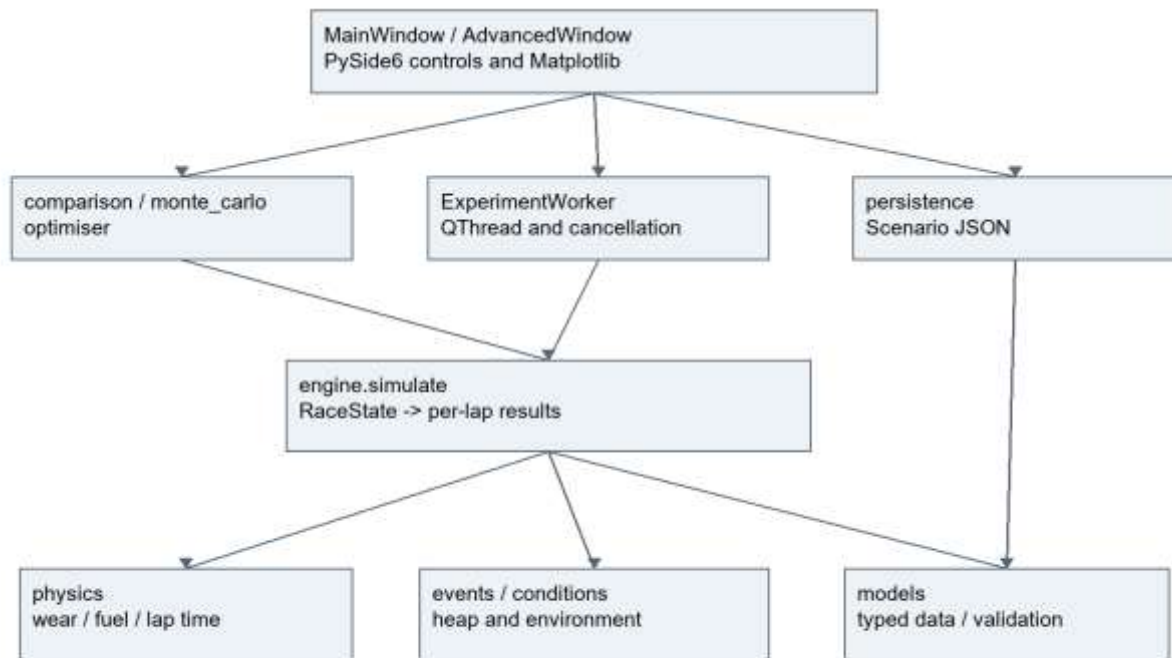


Figure 1: Implemented module architecture

2.3 Class design

The class inventory below is extracted from the current Python syntax tree. Constructor validation usually lives in `__post_init__`. Frozen dataclasses reduce accidental input mutation, although dictionaries inside `MonteCarloResult` remain mutable; frozen does not imply deep immutability.

Table 9: Classes in models

Class	Key attributes	Methods
Compound	Enum values or private queue state	Data record
Circuit	name: str; base_lap_time: float; pit_lane_loss: float	<code>__post_init__</code>
Driver	name: str; pace_delta: float	<code>__post_init__</code>
TyreCompound	name: Compound; base_pace_delta: float; degradation_rate: float; recommended_life: int	<code>__post_init__</code>
PitStopPlan	lap: int; compound: Compound; stationary_time: float	<code>__post_init__</code>
Strategy	name: str; starting_compound: Compound; planned_stops: tuple[PitStopPlan, ...]	<code>__post_init__</code> ; <code>validate_for</code>
RaceConfig	laps: int; circuit: Circuit; driver: Driver; tyres: tuple[TyreCompound, ...]; initial_fuel_penalty: float	<code>__post_init__</code> ; <code>tyre</code>
RaceState	completed_laps: int; compound: Compound; tyre_age: int; cumulative_time: float	<code>__post_init__</code>
LapResult	lap_number: int; lap_time: float; cumulative_time: float; compound: Compound; tyre_age: int; degradation_loss: float; fuel_effect: float; pit_stop: bool; pit_loss: float	<code>__post_init__</code>
StrategyResult	strategy: Strategy; laps: tuple[LapResult, ...]	<code>__post_init__</code> ; <code>total_time</code>

Table 10: Classes in events

Class	Key attributes	Methods
Weather	Enum values or private queue state	Data record
EventType	Enum values or private queue state	Data record
RaceEvent	lap: int; kind: EventType; weather: Weather None	__post_init__
EventQueue	Enum values or private queue state	__init__; at_lap

Table 11: Classes in conditions

Class	Key attributes	Methods
Conditions	initial_weather: Weather; events: tuple[RaceEvent, ...]; damp_penalty: float; wet_penalty: float; safety_car_penalty: float; safety_car_pit_factor: float; seed: int; variation: float	__post_init__; weather_loss
EnvironmentalLapResult	weather: Weather; weather_effect: float; safety_car: bool; safety_car_effect: float; random_variation: float	__post_init__

Table 12: Classes in monte_carlo

Class	Key attributes	Methods
Trial	seed: int; safety_car_start: int None; a_total: float; b_total: float	Data record
MonteCarloResult	trials: tuple[Trial, ...]; a_statistics: dict[str, float]; b_statistics: dict[str, float]; a_win_probability: float; tie_probability: float	Data record

Table 13: Classes in optimiser

Class	Key attributes	Methods
SearchSettings	max_stops: int; minimum_stint: int; pit_step: int; stationary_time: float	__post_init__
CandidateScore	strategy: Strategy; total_time: float	Data record

Table 14: Classes in persistence

Class	Key attributes	Methods
Scenario	config: RaceConfig; conditions: Conditions; strategies: tuple[Strategy, ...]	__post_init__

RaceConfig owns Circuit, Driver and the three TyreCompound definitions. Strategy contains PitStopPlan objects. StrategyResult refers to its Strategy and lap tuple. EnvironmentalLapResult inherits LapResult. Scenario joins config, conditions and strategies. AdvancedWindow extends MainWindow and owns ExperimentWorker. These are the relationships shown in the UML views in Appendix A.

2.4 Data structures

Table 15: Structures and alternatives

Structure and use	Reason	Alternative
-------------------	--------	-------------

Structure and use	Reason	Alternative
Tuple: tyres, stops, events and lap records	Fixed input/result sequences support reproducibility	Lists allow easier mutation but weaker boundaries
List: results, scores and trials during construction	Append then convert or sort	Preallocation adds bookkeeping without measured need
Dictionary: stops keyed by lap	Average constant-time stop lookup	Scanning all stops every lap
Tuple-keyed heap: EventQueue	Chronological order with sequence tie-break	Sort once for current static schedules
Enum: compound, weather, event type	Restricts valid named states	Unrestricted strings permit spelling errors
Dataclass: model and result records	Groups typed fields and validation	Parallel lists risk mismatched positions

RaceConfig.tyre actually searches its three-element tuple using next; it is not a dictionary lookup. With exactly three compounds this cost is bounded. This distinction matters when explaining the actual implementation.

2.5 Important variables

Table 16: Variables and units

Variable	Type and range	Purpose
laps	int, 1..500	Race distance
tyre_age	int, >=0	Completed laps on current set before this lap
base_lap_time	finite number >0, seconds	Reference racing pace
degradation_rate	finite number >=0, seconds per tyre lap	Linear wear slope
cumulative_time	finite number >=0, seconds	Accumulated total
seed	int, 0..2 ³² -1	Replayable generator state
variation	finite number >=0 and below base lap time	Uniform signed noise amplitude
safety_car_pit_factor	number 0..1	Relative pit-loss multiplier
minimum_stint / pit_step	int, 1..500	Search constraints
revision	integer counter in AdvancedWindow	Reject output for changed inputs

2.6 Algorithm design

The existing repository IDs are retained to avoid breaking traceability: A01 lap time, A02 degradation, A03 fuel, A04 strategy validation, A05 simulation, A06 events, A07 weather, A08 safety car, A09 comparison, A10 generation, A11 optimisation, A12 Monte Carlo, A13 statistics and A14 scenario validation. The numbering examples in the report brief were illustrative.

A01 and A03 Lap and fuel

Requirements FR05, FR06. Inputs: validated RaceConfig, lap index, current tyre, tyre age and pit loss. Output: lap time plus wear and fuel components. No random terms.

Recorded pseudocode or example

```

REQUIRE lap in [1, race.laps]
IF race.laps = 1 THEN fuel = 0
ELSE fuel = initial_fuel_penalty * (race.laps - lap) / (race.laps - 1)

```



```

wear = A02(tyre, age)
time = base_lap_time + driver_penalty + tyre_pace_penalty + wear + fuel + pit_loss
REQUIRE time is finite and positive
RETURN time, wear, fuel

```

Fuel is an illustrative time penalty decreasing linearly from the configured first-lap amount to zero on the final lap. The one-lap boundary uses zero. This abstracts consumed fuel without mass or burn-rate calibration. $O(1)$ time/space.

Planned tests TEST-I03: 5 laps, initial 2 gives fuel 2, 1.5, 1, .5, 0; 1 lap gives zero; invalid lap rejected. Base 90, driver 0, Soft pace 0, age 3 at .2, lap 2 fuel 1.5, pit 22.5 yields 114.6. Negative pit loss rejected. Alternative physical mass model is deferred.

A02 Tyre degradation

Requirement FR02; SC02. Purpose: return wear loss in seconds from a tyre's degradation rate and completed racing laps on that set.

Inputs: TyreCompound, integer age ≥ 0 . Output: non-negative finite seconds. Structure: scalar arithmetic, no stored state.

Recorded pseudocode or example

```

REQUIRE age is a non-negative integer
loss = tyre.degradation_rate * age
REQUIRE loss is finite
RETURN loss

```

$O(1)$ time and space. Linear model selected over quadratic/cliff alternatives because the current milestone has no empirical reason for complexity. Fresh tyre age is zero. Recommended life does not force a cliff.

Planned TEST-I02: rate .2 at ages 0, 1, 3 gives 0, .2, .6; zero-rate tyre remains zero; reject -1, fractional and boolean ages. Later engine tests must demonstrate resetting age after a stop.

A04 Strategy validation

Recorded pseudocode or example

```

REQUIRE race length is an integer in [1,500]
REQUIRE finite non-negative numeric model parameters
REQUIRE positive base lap time and positive recommended tyre life
REQUIRE exactly one definition for each supported compound
REQUIRE a supported starting compound
previous = 0
FOR each planned stop in supplied order
    REQUIRE previous < stop.lap < race length
    REQUIRE supported stop compound and non-negative stationary time
    previous = stop.lap
RETURN valid

```

Complexity $O(S + C)$, S stops and C tyre definitions. Do not sort invalid stop input silently.

Planned tests TEST-I01: valid defaults; races 1 and 500; reject 0, -1, 501, fractions, bool; invalid compound; duplicate/missing tyres; negative, non-finite parameters; missing start; duplicate, unordered and final-lap stops; invalid result objects. Expected outcomes are construction success or explanatory ValueError, not invented test results.

A05 Race and pit processing

Requirements FR03..FR06. These iterations are combined because applying a stop and verifying its following-lap effect requires a lap loop. The planned order is otherwise retained. Data: validated config and strategy, stop lookup by lap, RaceState and list of LapResult.

Recorded pseudocode or example

```

VALIDATE strategy against race

```

```

INDEX stops by lap
state = completed 0, starting compound, age 0, time 0
FOR lap = 1 TO race length
  stop = lookup stop for lap, if any
  pit_loss = lane loss + stationary time IF stop ELSE 0
  time, wear, fuel = A01(config, current tyre, age, lap, pit_loss)
  cumulative = state time + time
  APPEND LapResult using the tyre and age that ran this lap
  IF stop THEN next compound = stop compound AND next age = 0
  ELSE next compound = current compound AND next age = age + 1
  state = lap, next compound, next age, cumulative
RETURN StrategyResult

```

Time $O(N+S)$, space $O(N+S)$. Immutable state chosen for clear transitions, mutable state is a possible allocation optimisation only if measured. No events exist at this stage.

Independent manual oracle TEST-I05

Five laps, base 90, initial fuel penalty 2, lane loss 20, stationary 2.5. Soft pace 0/wear .2; Hard pace 1/wear .05. Stop after lap 2.

Table 17: Algorithm example or test data

Lap	Racing compound	Age	Wear	Fuel	Pit	Lap time	Cumulative
1	Soft	0	0	2	0	92	92
2	Soft	1	.2	1.5	22.5	114.2	206.2
3	Hard	0	0	1	0	92	298.2
4	Hard	1	.05	.5	0	91.55	389.75
5	Hard	2	.1	0	0	91.1	480.85

These are hand-derived expected values, not executed output. No-stop Soft expected 92, 91.7, 91.4, 91.1, 90.8 = 457.0 seconds.

Planned tests: manual oracle; repeated identical runs; immutable config; zero-stop one-lap race; first and penultimate lap stops; consecutive stops with same compound; invalid final stop; zero pit loss; 500-lap output count; total equals sum. Extreme finite inputs that overflow must fail descriptively rather than return infinity.

A06 Event queue

FR10. Implement the event data/queue separately before integration. Event types WEATHER_CHANGE, SAFETY_CAR_START, SAFETY_CAR_END. Weather states DRY, DAMP, WET. Input event: lap, type, optional weather value only for weather change. Queue key (lap, input_sequence) ensures chronological processing and deterministic same-lap ordering. Events apply before racing their lap, unlike pit stops which occur at the end.

Recorded pseudocode or example

```

FOR each event with input index:
  REQUIRE lap between 1 and race length
  REQUIRE correctly typed event payload
  PUSH (lap, input index, event) into heap
WHILE heap not empty:
  POP smallest key and yield event

```

$O(E \log E)$ time, $O(E)$ space. A sorted list would be simpler for static events; heap is chosen to support later dynamically scheduled events. Same-lap contradictory commands are applied in supplied order (last command determines state), documented rather than silently prioritised. Redundant SC starts/ends are idempotent state assignments.

Planned TEST-I08: unordered laps yield chronological order; ties preserve input sequence; lap 0/after race, invalid event type, wrong or missing weather payload rejected. Empty queue yields no events.

A07 Weather

FR10. Add Conditions: initial weather, immutable events, damp penalty 4 s, wet penalty 15 s (configurable illustrative assumptions, not measured data). All currently supported tyres are dry compounds. Wet/intermediate compounds remain optional and unimplemented, so wet weather penalises all dry tyres equally. This limits wet-weather strategy differentiation and must be visible in evaluation.

Add EnvironmentalLapResult extending LapResult with weather and weather_effect. Existing deterministic callers retain defaults and the same totals.

Recorded pseudocode or example

```
initial weather = configured state
BEFORE each lap: consume queue events; for weather change, assign new weather
penalty = 0 if Dry, configured damp penalty if Damp, configured wet penalty if Wet
ADD penalty to lap time before cumulative update
RECORD effective weather and penalty
```

$O(N + E \log E)$ time with constant-size weather lookup; $O(N+E)$ space. Alternative compound-specific wet suitability deferred until additional tyre types are justified. Planned TEST-I09: rain on lap 2, dry on lap 4 adds [0,15,15,0,0] to dry baseline; initial damp adds 4 each lap; same-lap wet then dry gives dry; event after race rejected. Dry regression total must remain identical.

A08 Safety car

FR10. Conditions adds safety_car_penalty (default 25 seconds/lap) and safety_car_pit_factor (default .5 in [0,1]). Both are configurable illustrative values. SC starts inactive. Start/end events before a lap toggle its state. Simultaneous commands use input order.

Recorded pseudocode or example

```
ON SC start: active = true
ON SC end: active = false
IF active:
    pit loss = (lane loss + stationary time) * pit factor
    lap safety penalty = configured penalty
ELSE lap safety penalty = 0
ADD safety penalty and record flag and penalty
```

This models relative pit opportunity, not field bunching, speed control or race neutralisation physics.

Weather/fuel/wear continue normally. $O(1)$ additional work per lap. Planned TEST-I10: SC starts 2 ends 4, adds 25 on laps 2 and 3; stop lap 2 costs 11.25 rather than 22.5. Net change vs dry stop oracle = $50 - 11.25 = 38.75$. Reject factor outside [0,1].

A09 Comparison

Requirements FR07/SC07. Input: one RaceConfig and 2..20 distinct-named strategies. Output: results ranked by ascending total time, stable input order for exact ties. Limit 20 is an interactive application bound, not a motorsport rule.

Recorded pseudocode or example

```
REQUIRE 2..20 strategies and unique names ignoring case and surrounding spaces
FOR each strategy: simulate using the same config
STABLE SORT results by total time
FOR each result:
    average = total / lap count
    fastest = minimum lap time
    slowest = maximum lap time
    total pit loss = sum of recorded pit loss
    stint lengths = successive differences of [0, stop laps..., race length]
RETURN sorted results with summaries
```

$O(K*N + K \log K)$ time and $O(K*N)$ result space. All lap summary metrics include pit laps; UI must say so. Alternative median racing-lap metric is deferred. Expected TEST-I06: manual Soft 457.0 beats manual pit strategy 480.85 by 23.85; average Soft 91.4; fastest 90.8; slowest 92; stint lengths [2,3]; equal strategies retain input order; duplicate names and too few/many strategies rejected.

A10 and A11 Search

FR12. Inputs: max stops 0..2, minimum stint ≥ 1 , pit-grid step ≥ 1 , stationary time, config and conditions. Generate compound products over Soft/Medium/Hard for each stint; combinations of eligible pit laps enforce increasing stops. Include 0-stop strategies. Grid begins at minimum stint and ends N-minimum stint. All consecutive stint lengths must satisfy the minimum, including final stint. Reject minimum stint $> N$.

Recorded pseudocode or example

```
FOR stops from 0 to maximum:
  FOR increasing stop-lap combination from grid:
    IF every stint has minimum length:
      FOR every compound tuple of length stops+1:
        YIELD legal Strategy
FOR candidate:
  check candidate/work caps BEFORE simulating
  simulate and STORE candidate plus total
STABLE SORT scores by total; return scores and count
```

Caps: 10,000 candidate scores and two million lap evaluations. Abort descriptively if exceeded; never silently present a partial search as complete. Bounds stop generation before large allocation. Cancellation callback each candidate. Conditions noise is set to zero for search: result is deterministic under fixed events, not a stochastic optimum.

Time $O(C*N + C \log C)$, memory $O(C*S + N)$; compounds contribute $3^{(S+1)}$. Candidate generation uses itertools combinations/product (explainable nested iteration). Baselines: Hard no-stop, Medium->Hard halfway when legal, Medium renewed at its recommended life up to two stops. Baselines may fall outside search grid and must be identified as separately evaluated reference rules.

Planned TEST-I13: $N=3$, min1, max1, grid1 yields $3+2*9=21$ unique candidates; stop bounds and minimum stints; $N=2$ under manual config has best Soft no-stop at 182.2 s; independent exhaustive candidate totals must match ranking; cap failure rather than partial answer; cancellation; zero-noise search matches input conditions replaced with amplitude0. Alternative heuristic faster but lacks complete bounded enumeration.

A12 and A13 Repeated trials and statistics

FR11. Input: two distinct strategies, config, conditions, run count 1..1000 (GUI 100/500/1000), safety-car occurrence probability [0,1], duration ≥ 1 . Work cap two million racing laps per request. Store per-trial seed, generated SC lap or None, and paired totals. Summary: mean, median, population standard deviation, min, max, A-win probability and tie probability. Ties use absolute $1e-9$ second tolerance to avoid floating noise.

Recorded pseudocode or example

```
validate inputs and both strategies before loop
scheduler = independent Random(master seed)
FOR trial:
  draw trial seed
  with configured probability, draw SC start uniformly in [1,N]
  append generated SC start/end to fixed events (generated ties come last)
  simulate A and B using identical trial conditions
  store both totals and seed and generated start
summarise samples; count wins and ties; return audit data
```

No generated rain initially: fixed weather schedule remains reproducible. Common lap disturbances alone cancel; random SC timing changes relative pit opportunity. Generated SC may overlap fixed SC events: all commands are assignments in documented order, not independently stacked safety-car periods. GUI/user documentation must state this limitation.

$O(R \cdot (N + E \log E))$, $O(R + N + E)$ working space by discarding full lap results after each total. Cancellation callback checked each trial. Alternative independent noise would create ranking variability from unrelated draws; common conditions chosen for fair pairing.

Planned TEST-I12: stats [1,2,3] mean/median 2, population SD $\sqrt{2/3}$, min1/max3; zero-uncertainty repeated totals; seed exact replay including SC events; identical strategies tie probability1 even with noise; invalid counts/probability/work rejected; count100 actual smoke before larger benchmark counts.

A14 Scenario persistence

FR09. Order change: persistence before final GUI/charts so save/load integrates once. Scenario schema version1 stores config, all tyre inputs, driver/circuit names, conditions including seed and events, and 1..20 strategies including each stop's stationary time. Search/Monte Carlo run controls are not part of the racing scenario; their result audit records will retain them separately.

Use dataclass-to-dict and explicit Enum values. Decode only exact expected fields, then construct validated dataclasses and validate all strategies/events against race length. Reject bool schema versions, unsupported versions, duplicate JSON keys, missing/extra fields, nonfinite JSON constants, files over1MiB and too many strategies/events. Max500 events. No pickle or executable objects.

Recorded pseudocode or example

SAVE: validate scenario; encode JSON with nonfinite values disabled
write sibling temporary file; atomically replace requested destination
LOAD: check byte size; parse JSON rejecting duplicates/nonfinite
check schema version and exact keys recursively
construct and validate typed objects
return complete Scenario only after all checks pass

Atomic write reduces partial-save risk. Destination replacement is only through user-selected save or explicit scripting. $O(\text{file bytes} + \text{event count} + \text{stop count})$. Alternative database unnecessary for small portable scenarios.

Planned TEST-I15: exact round-trip includes events, seed, compounds and stop service time; malformed JSON, missing keys, duplicate keys, bad version, NaN, negative length, unsupported compound, duplicate stops, event after race, oversized file rejected; failed load cannot modify live GUI state.

Current-code refinement: EventQueue builds a list and heapifies it in $O(E)$, rather than pushing every item separately. Consuming all events is $O(E \log E)$ [R04]. A race takes $O(N + E \log E)$ with fixed three-compound lookup. Monte Carlo also sorts samples for median, adding $O(R \log R)$ summary work [R05, R08]. Candidate search adds $O(C \log C)$ sorting after C race simulations. These are analyses of the actual code, not measured timing formulas.

Seeded variation uses a private Random(seed), one uniform draw per lap, and a zero-noise shortcut. Equal seeds reproduce results in the recorded environment. Because the same additive draw is used for both strategies, noise alone cancels in their gap; random safety-car timing changes pit opportunity. Pit processing occurs after racing lap L, while environmental events are applied before it.

2.7 Validation design

Table 18: Core data responsibilities

Class	Responsibility and fields
Compound (enum)	SOFT, MEDIUM, HARD identifiers
Circuit	name, base_lap_time (seconds), pit_lane_loss (seconds)
Driver	name, pace_delta (non-negative seconds relative to circuit baseline)
TyreCompound	name, base_pace_delta (non-negative seconds), degradation_rate (seconds per completed tyre lap), recommended_life (positive laps)

Class	Responsibility and fields
PitStopPlan	lap (pit at end of lap), compound, stationary_time (seconds)
Strategy	name, starting_compound, ordered immutable tuple of stops
RaceConfig	laps (1..500), circuit, driver, three tyre definitions, initial_fuel_penalty (seconds)
RaceState	completed_laps, compound, tyre_age, cumulative_time
LapResult	lap_number, lap_time, cumulative_time, compound used for racing lap, tyre_age, degradation_loss, fuel_effect, pit_stop, pit_loss
StrategyResult	strategy, immutable lap results, derived total_time

Input objects validate on construction. Cross-object strategy validation checks final race length. Result/state objects validate types, bounds and finite times; cross-lap correctness is tested at the engine level. Floats reject NaN and infinity, negative values and booleans. Integers reject booleans. Tuple collections prevent caller mutation after validation.

Presence checks reject empty names. Type checks reject boolean race lengths even though bool is an int subclass in Python. Range checks reject negative and nonfinite values. Lookup checks require supported enums and exactly one tyre definition of each type. Logical checks reject duplicated, unordered and final-lap stops. JSON checks require exact fields and arrays before validating elements. The final collection-type defect and fix are documented in Section 3.16.

2.8 Usability design

Requirements FR01..FR08. PySide6 widgets, Matplotlib Qt canvas. No decorative imagery. Dark panels, high contrast text, labelled inputs with units, compound text plus colour, read-only output tables, explicit pending/invalid/result status. Keyboard-operable Qt controls.

Planned layout: title and model disclaimer; left race and tyre inputs; centre/right chart, ranked summary and lap table tabs; strategy editor below chart; run-selected and compare-all buttons. Changing inputs invalidates old output immediately so stale results cannot appear current. Strategy selection in output chooses whose lap rows are shown.

Data conversion: numeric widgets -> validated config; strategy rows -> Strategy. Stop entry syntax 15:Medium, 35:Hard with a shared configurable stationary time. Blank means no stops. Parser accepts case-insensitive supported compound names, rejects missing separators, unknown compounds and invalid laps; model validates ordering and race boundary. No silent reordering. Names unique for comparisons.

Recorded pseudocode or example

```
ON input change: clear old output and request new simulation
ON run:
    READ and VALIDATE configuration
    READ selected strategy or all strategies
    TRY simulation/comparison
    ON validation error: clear output and show explanatory status
    ON success: plot lap/time from returned records, populate ranking
                select first result and populate per-lap table
```

Synchronous execution is limited to 500 laps and 20 strategies. Later Monte Carlo/search must consider a worker and cancellation. GUI exposes only implemented actions.

Planned TEST-I07: parser typical/blank/erroneous cases; instantiate real window; compare defaults and assert 2 lines of 50 points; select single strategy and assert 50 lap rows; modify race length and ensure stale output clears; malformed stop -> visible error and no output; add/remove strategies; capture actual Qt widget output. This is automated integration/visual evidence, not a real stakeholder usability session.

Manual post-development plan: genuine user opens app, configures five laps, creates two strategies, runs, identifies faster total and pit spike, changes parameters, corrects a bad stop. Record completion, errors, help, comments only when performed by a user. Test keyboard navigation and display scaling on their machine. No responses yet.

2.9 Current interface design

No historical wireframe file was found. The following actual screenshot documents the implemented interface. It is not retrospective design evidence. Tabs separate comparison, per-lap detail, conditions and experiments. Input changes clear stale results; background work uses snapshots and revision checks. Text syntax saves space but needs genuine user evaluation.

Figure 2 shows the default two-strategy comparison and its pit-lap spikes. The totals are model outputs, not real-race observations.

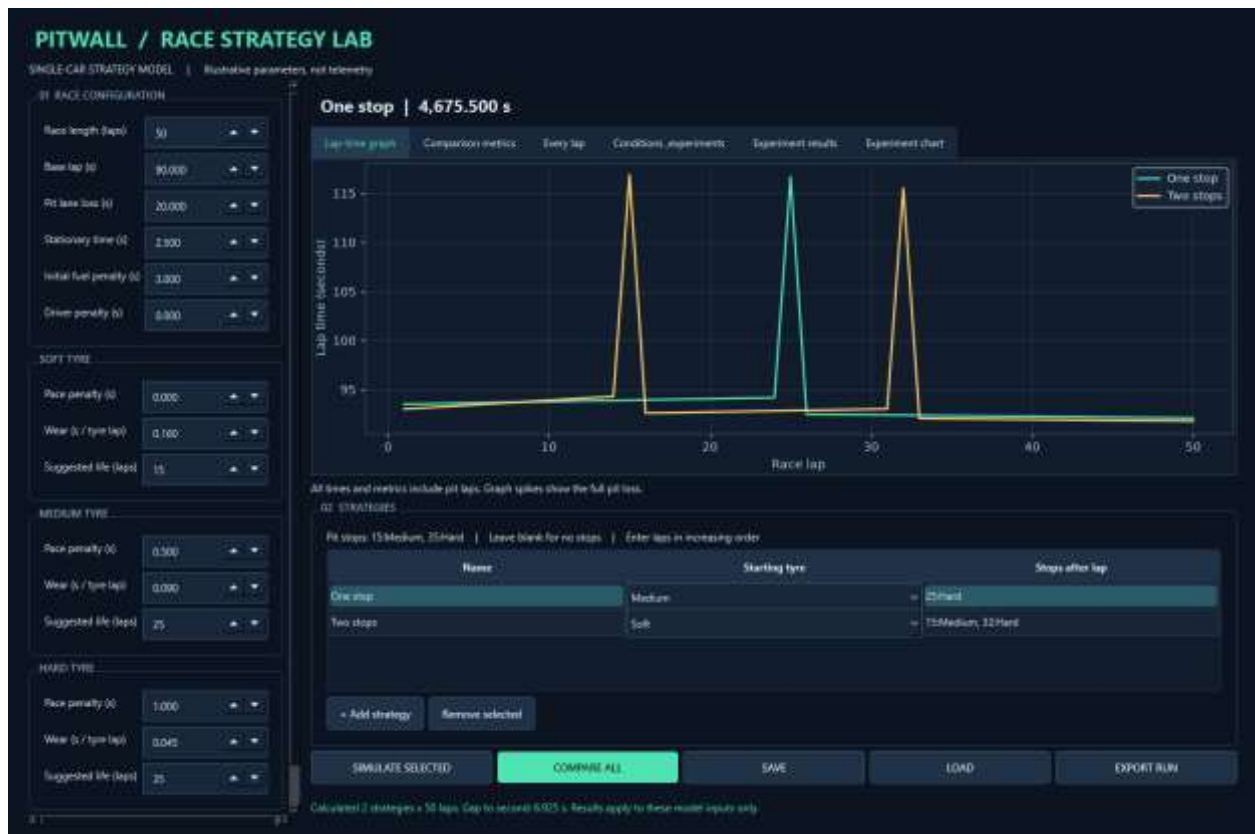


Figure 2: Current strategy comparison interface

2.10 Iterative test plan

Table 19: Typical boundary and erroneous tests

Area	Typical	Boundary	Erroneous and purpose
Race input	5 laps	1 and 500	0,501,bool: enforce finite scope
Tyre wear	age 3,rate0.2 ->0.6	age 0 or zero rate	negative/fractional age
Stops	lap 2 in five laps	first and penultimate laps	duplicate,unordered,final stop
Events	rain then dry	same-lap sequence	missing payload/outside race
Monte Carlo	100 trials,seed42	one trial,zero noise	0,1001,invalid seed
JSON	full round-trip	1 MiB bound	malformed,duplicates,wrong arrays

2.11 Post-development test plan

Do not invent failures. The runner preserves every executed result with timestamp and source hashes before a fix.

Separate post-development tests will use different inputs from iterative tests: malformed collection types in JSON; bool and nonfinite settings; 500-lap completed comparison; 20-strategy limit; independent small exhaustive search oracle; random SC replay by reconstructing one trial; stale worker output rejection; complete advanced GUI capture with actual results. These are automated final checks, not human usability tests.

Performance: record Python/OS/CPU/RAM and raw perf_counter trials for one race, 100/500/1000 sequential races, 10/100/1000 candidate-scoring prefixes and full default search, 100/500/1000 paired Monte Carlo trials. Three measured repeats each after one single-race warm-up. Include minimum, maximum, mean, median. Prefix scoring benchmarks measure throughput, not completed optimisation. Timings include Python result construction and omit GUI rendering. Machine workload/power may vary; no cross-machine guarantee.

Prospective target for this development machine: default comparison <1s, default bounded search <10s and 1000 paired 50-lap trials <30s; evaluate only after measurement. Cancellation should return promptly between laps/candidates/trials; no hard real-time guarantee. No 5000-trial option until tested and justified.

Section 4 reports the preserved executions separately from this plan. Section 4.6 separately records the new final-quality test executions and comparison measurements. Human usability tasks remain planned in Appendix F.

3 Development and testing

The recorded sequence is I01, I02, I03, combined I04/I05, I06, I07, I08, I09, I10, I11, I12, I13, I15, combined I14/I16, I17. Pit integration was combined with the full loop; persistence preceded the final GUI/chart integration. Seven genuine commits cover these stages rather than one commit per iteration. All recorded implementation commits are on 8 September 2026; this report does not invent a multi-week history. Appendix G lists the exact commit timestamps.

3.1 Core models

Objective and links. I01: Core models. Requirements FR01 FR02 FR03 FR04; success criteria SC01 SC02 SC03 SC04. Design: docs/design/architecture.md.

Implementation and recorded review. The following preserved iteration account records the implementation, expected and actual results, alternatives and next step. Its claims are bounded by the linked execution evidence.

Objective: FR01..FR04. Design and pseudocode: docs/design/architecture.md. Files: src/pitwall/models.py, tests/conftest.py, tests/test_models.py, scripts/run_tests.py.

Implementation uses frozen dataclasses, a compound enum, finite numeric checks, immutable tuple collections and race-dependent strategy validation. Strategy.validate_for checks the final-lap constraint; constructor validation rejects duplicates and unordered stops rather than silently sorting them. This separates local object validity from race context.

Tests: TEST-I01 executed 2026-09-08 UTC. Expected: valid inputs accepted and erroneous inputs raise ValueError. Actual: 20 passed. Raw output, JUnit XML and source hashes are in evidence/test-runs/TEST-I01-20260908T205847968425Z/.

Genuine test failures: none observed. Diagnosis, remedial action and retest: N/A. Environment issue: dependency installation failed at a PySide6 long pathname; a shorter runtime path is being prepared. This is an installation problem, not a manufactured code-test failure. Original tool output remains in the task conversation.

Screenshot evidence: pending, no GUI yet. [GENUINE STAKEHOLDER FEEDBACK PENDING]

Review: boundary validation passed; the cross-object final-lap rule required an explicit convention. An alternative mutable model would simplify edits but permit invalid states after construction. Lesson: validate newly constructed inputs at the GUI boundary. Implementation matches the recorded class design. Candidate reflection and explanation are pending. Next: tyre degradation.

Code extract: src/pitwall/models.py, lines 15-17 (complete integer)

```
15: def integer(value: int, name: str, minimum: int = 0, maximum: int | None = None) -> None:
16:     if type(value) is not int or value < minimum or (maximum is not None and value > maximum):
17:         raise ValueError(f"{name} must be an integer from {minimum} to {maximum if maximum is
not None else 'unbounded'}")
```

Code explanation. The strict type check excludes bool and fractions before range validation. This prevents misleading race lengths from reaching simulation.

Extract provenance. This is the current-source version, with line numbers, not a claim that every later extension existed at this earlier stage. The preserved run metadata identifies the source hashes used for each actual test run.

Stakeholder review. [GENUINE STAKEHOLDER REVIEW REQUIRED]

Candidate reflection. [CANDIDATE REVIEW AND EXPLANATION REQUIRED]

3.2 Tyre degradation

Objective and links. I02: Tyre degradation. Requirements FR02; success criteria SC02. Design: docs/design/I02-tyres.md.

Implementation and recorded review. The following preserved iteration account records the implementation, expected and actual results, alternatives and next step. Its claims are bounded by the linked execution evidence.

Objective FR02/SC02; design and A02 pseudocode: docs/design/I02-tyres.md. Changed physics.py and added test_physics.py. Multiplication by completed tyre age keeps units and the fresh-tyre convention explicit. Validation rejects invalid ages and non-finite calculated loss.

Expected: ages 0, 1, 3 at .2 give 0, .2, .6 seconds; bad ages rejected. Actual: all 27 accumulated tests passed in TEST-I02, including seven new cases. Evidence: evidence/test-runs/TEST-I02-20260908T205945777891Z/. No observed test failure; diagnosis/fix/retest N/A. Screenshots N/A for this pure function. [GENUINE STAKEHOLDER FEEDBACK PENDING]

Review: simple arithmetic matched manual expectations. Floating-point assertions require tolerance because .2 times 3 is not exactly representable. Quadratic wear remains an alternative requiring justification. No design deviation. Candidate reflection pending. Next: deterministic lap and fuel calculation.

Code extract: src/pitwall/physics.py, lines 5-9 (complete degradation)

```
5: def degradation(tyre: TyreCompound, age: int) -> float:
6:     integer(age, "Tyre age")
7:     loss = tyre.degradation_rate * age
8:     number(loss, "Degradation loss")
9:     return loss
```

Code explanation. The rate multiplies completed tyre age; validation checks the input age and the finite calculated loss.

Extract provenance. This is the current-source version, with line numbers, not a claim that every later extension existed at this earlier stage. The preserved run metadata identifies the source hashes used for each actual test run.

Stakeholder review. [GENUINE STAKEHOLDER REVIEW REQUIRED]

Candidate reflection. [CANDIDATE REVIEW AND EXPLANATION REQUIRED]

3.3 Lap and fuel calculation

Objective and links. I03: Lap and fuel calculation. Requirements FR05 FR06; success criteria SC05 SC06. Design: docs/design/I03-lap.md.

Implementation and recorded review. The following preserved iteration account records the implementation, expected and actual results, alternatives and next step. Its claims are bounded by the linked execution evidence.

Objective FR05/FR06; A01/A03 design: docs/design/I03-lap.md. Changed physics and its tests. Fuel is a linear seconds penalty, not physical fuel mass. The single-lap boundary avoids division by zero.

Expected: fuel sequence 2, 1.5, 1, .5, 0 and manual lap 114.6 seconds. Actual: 33 accumulated tests passed, including six new tests. Evidence: evidence/test-runs/TEST-I03-20260908T210046769795Z/. No observed test failures; diagnosis/remedy/retest N/A. Screenshot N/A. [GENUINE STAKEHOLDER FEEDBACK PENDING]

Review: component outputs make later diagnosis possible. The one-lap fuel convention is a modelling choice, not a real-world claim. Alternative mass/burn-rate model deferred. No design deviation. Candidate reflection pending. Next: integrate pit stops and full race simulation.

Code extract: src/pitwall/physics.py, lines 12-16 (complete fuel_effect)

```
12: def fuel_effect(config: RaceConfig, lap: int) -> float:
13:     integer(lap, "Lap number", 1, config.laps)
14:     if config.laps == 1:
15:         return 0.0
16:     return config.initial_fuel_penalty * ((config.laps - lap) / (config.laps - 1))
```

Code explanation. The one-lap branch avoids division by zero. Other races decrease the initial penalty linearly to zero.

Extract provenance. This is the current-source version, with line numbers, not a claim that every later extension existed at this earlier stage. The preserved run metadata identifies the source hashes used for each actual test run.

Stakeholder review. [GENUINE STAKEHOLDER REVIEW REQUIRED]

Candidate reflection. [CANDIDATE REVIEW AND EXPLANATION REQUIRED]

3.4 Pit stops and dry race

Objective and links. I04-I05: Pit stops and dry race. Requirements FR03 FR05 FR06; success criteria SC03 SC05 SC06. Design: docs/design/I04-I05-engine.md.

Implementation and recorded review. The following preserved iteration account records the implementation, expected and actual results, alternatives and next step. Its claims are bounded by the linked execution evidence.

Objective FR03..FR06; design/A05 and independent expected arithmetic: docs/design/I04-I05-engine.md. These two stages were combined before implementation because pit-stop effects span adjacent laps. Added engine.py and test_engine.py.

The loop records the old compound on the pit lap, adds lane and stationary loss once, then changes the state for the next lap. Dictionary lookup avoids scanning all stops every lap. Every lap retains wear and fuel components.

Actual TEST-I05: 41 accumulated tests passed, including eight new tests. Manual stop race total matched 480.85 seconds; no-stop Soft matched 457.0. Evidence: evidence/test-runs/TEST-I05-20260908T210214046535Z/. Invalid final stops and overflowing results were rejected as expected. These expected rejection cases are passing tests, not genuine failures.

No observed failed tests. Diagnosis/remedy/retest N/A. Screenshots pending GUI. [GENUINE STAKEHOLDER FEEDBACK PENDING]

Review: indexing is now explicit and verified at both ends of the race. Immutable states favour clarity over avoiding allocations. Alternative mutable state is deferred until profiling. Design followed including combined order. Candidate reflection pending. Next: comparison.

Code extract: src/pitwall/engine.py, lines 9-24 (selected opening simulate)

```
9: def simulate(config: RaceConfig, strategy: Strategy, conditions: Conditions | None = None) ->
StrategyResult:
10:     strategy.validate_for(config)
11:     conditions = conditions or Conditions()
12:     if conditions.variation >= config.circuit.base_lap_time:
13:         raise ValueError("Variation amplitude must be below base lap time")
14:     rng = random.Random(conditions.seed)
15:     queue = EventQueue(conditions.events, config.laps)
16:     weather = conditions.initial_weather
17:     safety_car = False
18:     stops = {stop.lap: stop for stop in strategy.planned_stops}
19:     state = RaceState(0, strategy.starting_compound, 0, 0.0)
20:     results = []
21:     for lap in range(1, config.laps + 1):
22:         for event in queue.at_lap(lap):
23:             if event.kind == EventType.WEATHER_CHANGE:
24:                 weather = event.weather
```

Code explanation. Validation occurs before iteration. State and stop lookup are local to each run; events are processed at the start of a lap.

Extract provenance. This is the current-source version, with line numbers, not a claim that every later extension existed at this earlier stage. The preserved run metadata identifies the source hashes used for each actual test run.

Stakeholder review. [GENUINE STAKEHOLDER REVIEW REQUIRED]

Candidate reflection. [CANDIDATE REVIEW AND EXPLANATION REQUIRED]

3.5 Strategy comparison

Objective and links. I06: Strategy comparison. Requirements FR07; success criteria SC07. Design: docs/design/I06-comparison.md.

Implementation and recorded review. The following preserved iteration account records the implementation, expected and actual results, alternatives and next step. Its claims are bounded by the linked execution evidence.

FR07/SC07 and A09: docs/design/I06-comparison.md. Added comparison module and tests. Comparison shares one config and sorts by total with stable input-order ties. Summaries include pit laps, and stint lengths follow the end-of-lap stop convention.

Expected: Soft no-stop beats pit strategy by 23.85 seconds, average 91.4, fastest 90.8, slowest 92; stop strategy stints (2,3). Actual: 44 accumulated tests passed, including three new tests. Evidence: evidence/test-runs/TEST-I06-20260908T210320581715Z/. No observed failures, diagnosis/remedy/retest N/A. Screenshot pending GUI.

[GENUINE STAKEHOLDER FEEDBACK PENDING]

Review: stable ranking avoids surprising tie reordering. Name uniqueness removes ambiguity in graphs; 20-strategy cap bounds interactive output. Median racing-lap metrics are a possible alternative. No design deviation. Candidate reflection pending. Next: desktop interface with real graphs.

Code extract: src/pitwall/comparison.py, lines 18-30 (complete summary)

```
18: def summary(result: StrategyResult) -> dict:
19:     times = [lap.lap_time for lap in result.laps]
20:     boundaries = [0, *(s.lap for s in result.strategy.planned_stops), len(times)]
21:     return {
22:         "name": result.strategy.name,
23:         "total": result.total_time,
24:         "stops": len(result.strategy.planned_stops),
25:         "average": result.total_time / len(times),
26:         "fastest": min(times),
27:         "slowest": max(times),
28:         "pit_loss": sum(lap.pit_loss for lap in result.laps),
29:         "stints": tuple(b - a for a, b in zip(boundaries, boundaries[1:])),
30:     }
```

Code explanation. Metrics come from recorded lap times and stop boundaries. Pit laps remain included in fastest, slowest and average values.

Extract provenance. This is the current-source version, with line numbers, not a claim that every later extension existed at this earlier stage. The preserved run metadata identifies the source hashes used for each actual test run.

Stakeholder review. [GENUINE STAKEHOLDER REVIEW REQUIRED]

Candidate reflection. [CANDIDATE REVIEW AND EXPLANATION REQUIRED]

3.6 First desktop milestone

Objective and links. I07: First desktop milestone. Requirements FR01 FR08; success criteria SC01 SC08. Design: docs/design/I07-gui.md.

Implementation and recorded review. The following preserved iteration account records the implementation, expected and actual results, alternatives and next step. Its claims are bounded by the linked execution evidence.

Objective FR01..FR08; GUI design, validation and test plan: docs/design/I07-gui.md. Added defaults, GUI, launcher, GUI tests and actual-window capture script. Plots are populated directly from engine LapResult records. Input edits clear stale output. Error status explains invalid stop syntax and race-dependent constraints. Only implemented actions are exposed.

Expected: two 50-point graph lines, two ranked rows, 50 lap rows, cleared output after editing, recoverable input errors. Actual: 48 accumulated tests passed, including four GUI/parser tests. Evidence: evidence/test-runs/TEST-I07-20260908T210702638724Z/.

Actual Windows Qt capture: evidence/screenshots/FIG-I07-20260908T210757444690Z/application.png; adjacent capture.json records method, UTC timestamp and actual results. Visual inspection confirmed that the chart, strategy rows and controls rendered. A scroll area exposes settings that exceed available height. This is a capture of a running application, not a mock-up, desktop screenshot or human usability session.

Default actual totals: One stop 4675.5 s; Two stops 4682.425 s (floating representation may differ in raw output). Gap 6.925 s. These are fictional-parameter model outputs, not real motorsport predictions.

No observed test failures; diagnosis/remedy/retest N/A. Environment workaround succeeded: runtime in the user's temporary directory avoids PySide6 long paths. [GENUINE STAKEHOLDER FEEDBACK PENDING]

Review: readable chart and explicit stop convention support inspection. Text stop entry is compact but genuine users may prefer a dedicated stop editor. Keyboard/display-scaling usability remains untested. Implementation follows design. Candidate review and personal reflection pending. First deterministic milestone achieved; next: events.

Code extract: src/pitwall/gui.py, lines 19-33 (complete parse_stops)

```
19: def parse_stops(text: str, stationary: float) -> tuple[PitStopPlan, ...]:
20:     if not text.strip():
21:         return ()
22:     stops = []
23:     for entry in text.split(','):
24:         try:
25:             parts = entry.split(':')
26:             if len(parts) not in (2, 3):
27:                 raise ValueError("Expected lap:tyre[:stationary]")
28:             lap, tyre = parts[:2]
29:             service = float(parts[2]) if len(parts) == 3 else stationary
30:             stops.append(PitStopPlan(int(lap.strip()), Compound(tyre.strip().capitalize()),
service))
31:         except (ValueError, TypeError) as exc:
32:             raise ValueError(f"Invalid stop '{entry.strip()}'". Use 15:Medium, 35:Hard with
positive lap numbers.") from exc
33:     return tuple(stops)
```

Code explanation. The parser converts text into typed stops; Strategy validation then checks ordering and the race boundary.

Extract provenance. This is the current-source version, with line numbers, not a claim that every later extension existed at this earlier stage. The preserved run metadata identifies the source hashes used for each actual test run.

Figure 3 shows the actual first-milestone comparison before the advanced controls were integrated.

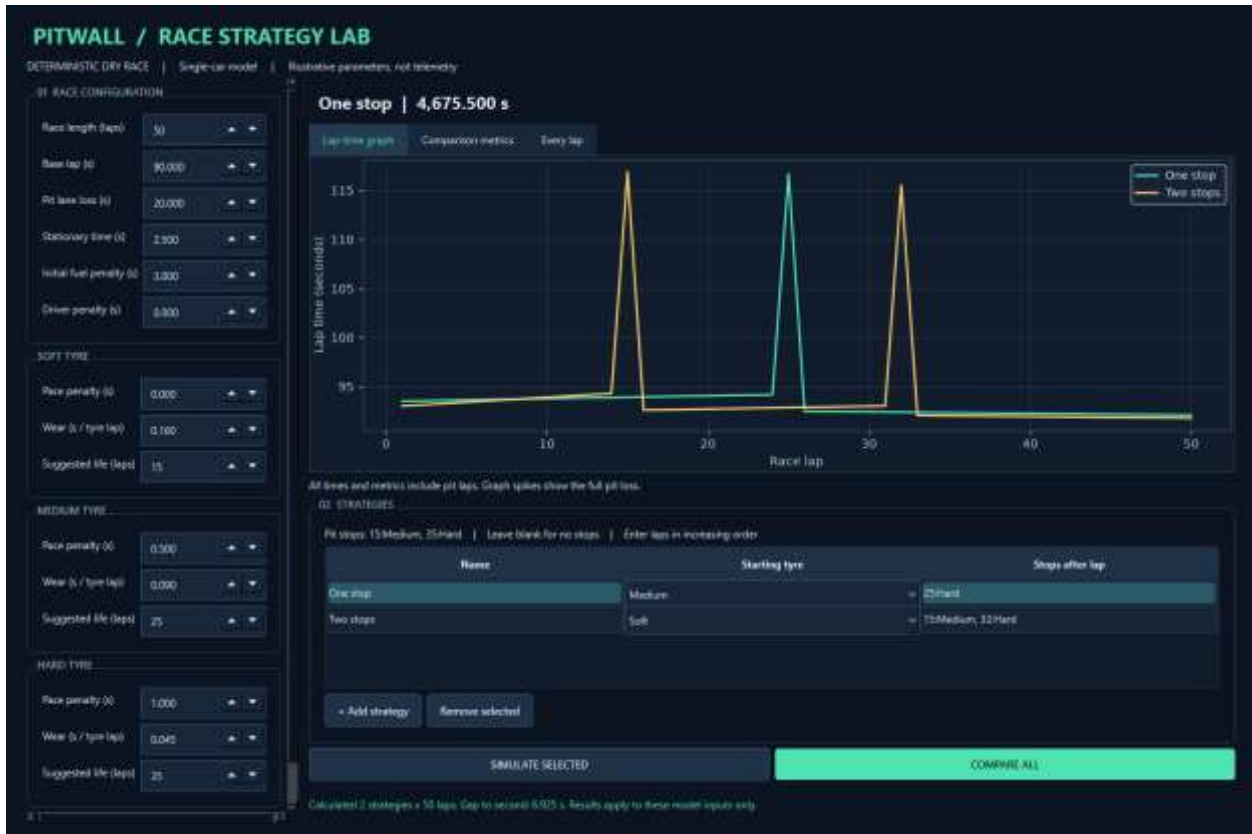


Figure 3: First deterministic GUI milestone

Stakeholder review. [GENUINE STAKEHOLDER REVIEW REQUIRED]

Candidate reflection. [CANDIDATE REVIEW AND EXPLANATION REQUIRED]

3.7 Event queue

Objective and links. I08: Event queue. Requirements FR10; success criteria SC10. Design: docs/design/I08-events.md.

Implementation and recorded review. The following preserved iteration account records the implementation, expected and actual results, alternatives and next step. Its claims are bounded by the linked execution evidence.

FR10/A06 design docs/design/I08-events.md. Added events.py and test_events.py. Queue keys are lap and supplied sequence number; payloads validate independently. Expected chronological order and stable ties matched actual output. TEST-I08: 2 focused tests passed; evidence evidence/test-runs/TEST-I08-20260908T210944391018Z/. These test functions include several invalid-input assertions.

No observed failures; remedy/retest N/A. Screenshot N/A. [GENUINE STAKEHOLDER FEEDBACK PENDING]

Review: heap is appropriate for future insertion but current static lists could simply be sorted. Integration with lap state is next. No design deviation; candidate reflection pending.

Code extract: src/pitwall/events.py, lines 37-50 (complete EventQueue)

```

37: class EventQueue:
38:     def __init__(self, events: tuple[RaceEvent, ...], laps: int):
39:         if not isinstance(events, tuple) or not all(isinstance(e, RaceEvent) for e in events):
40:             raise ValueError("Events must be a tuple of RaceEvent objects")
41:         for event in events:
42:             integer(event.lap, "Event lap", 1, laps)
43:         self._heap = [(event.lap, index, event) for index, event in enumerate(events)]
44:         heapq.heapify(self._heap)
45:
46:     def at_lap(self, lap: int) -> tuple[RaceEvent, ...]:
47:         due = []
48:         while self._heap and self._heap[0][0] <= lap:
49:             due.append(heapq.heappop(self._heap)[2])
50:         return tuple(due)

```

Code explanation. The sequence number prevents ambiguous same-lap ordering and avoids comparing RaceEvent objects.

Extract provenance. This is the current-source version, with line numbers, not a claim that every later extension existed at this earlier stage. The preserved run metadata identifies the source hashes used for each actual test run.

Stakeholder review. [GENUINE STAKEHOLDER REVIEW REQUIRED]

Candidate reflection. [CANDIDATE REVIEW AND EXPLANATION REQUIRED]

3.8 Weather effects

Objective and links. I09: Weather effects. Requirements FR10; success criteria SC10. Design: docs/design/I09-weather.md.

Implementation and recorded review. The following preserved iteration account records the implementation, expected and actual results, alternatives and next step. Its claims are bounded by the linked execution evidence.

FR10/A07 design docs/design/I09-weather.md. Added Conditions and EnvironmentalLapResult; integrated weather queue into engine. TEST-I09 ran 12 engine/event/weather tests, all passed. Evidence evidence/test-runs/TEST-I09-20260908T211102758207Z/. Expected wet increment 30 s and damp increment 20 s matched manual arithmetic; dry engine regressions passed.

No observed failure; remedy/retest N/A. Screenshot pending updated GUI. [GENUINE STAKEHOLDER FEEDBACK PENDING]

Review: inherited results preserve original lap fields. Weather currently penalises dry compounds equally, so no claim of realistic wet tyre selection. Optional wet/intermediate tyres deferred. Same-lap event order behaves as designed. Candidate reflection pending; next safety car.

Code extract: src/pitwall/conditions.py, lines 32-33 (complete weather_loss)

```
32:     def weather_loss(self, weather: Weather) -> float:
33:         return {Weather.DRY: 0.0, Weather.DAMP: self.damp_penalty, Weather.WET:
self.wet_penalty}[weather]
```

Code explanation. An explicit lookup selects the environmental time penalty. It does not model a wet tyre advantage.

Extract provenance. This is the current-source version, with line numbers, not a claim that every later extension existed at this earlier stage. The preserved run metadata identifies the source hashes used for each actual test run.

Stakeholder review. [GENUINE STAKEHOLDER REVIEW REQUIRED]

Candidate reflection. [CANDIDATE REVIEW AND EXPLANATION REQUIRED]

3.9 Safety car

Objective and links. I10: Safety car. Requirements FR10; success criteria SC10. Design: docs/design/I10-safety-car.md.

Implementation and recorded review. The following preserved iteration account records the implementation, expected and actual results, alternatives and next step. Its claims are bounded by the linked execution evidence.

FR10/A08 design docs/design/I10-safety-car.md. Conditions, environmental results, engine and tests changed. Starts/ends assign state before racing the lap. Pit factor represents relative lost time, not actual service duration.

TEST-I10: 16 focused regression/event/condition tests passed. Evidence evidence/test-runs/TEST-I10-20260908T211214733077Z/. Manual expected total 519.6 s and pit loss 11.25 s matched. No observed failure; remedy/retest N/A. Screenshot pending integration. [GENUINE STAKEHOLDER FEEDBACK PENDING]

Review: explicit simplified terms remain inspectable. Real bunching would need multiple cars and is beyond this scope. No design deviation; candidate reflection pending. Next seeded lap variation.

Code extract: `src/pitwall/conditions.py`, lines 9-24 (selected opening Conditions)

```

9: class Conditions:
10:     initial_weather: Weather = Weather.DRY
11:     events: tuple[RaceEvent, ...] = ()
12:     damp_penalty: float = 4.0
13:     wet_penalty: float = 15.0
14:     safety_car_penalty: float = 25.0
15:     safety_car_pit_factor: float = 0.5
16:     seed: int = 0
17:     variation: float = 0.0
18:
19:     def __post_init__(self):
20:         if not isinstance(self.initial_weather, Weather):
21:             raise ValueError("Initial weather must be Dry, Damp or Wet")
22:         EventQueue(self.events, 500)
23:         number(self.damp_penalty, "Damp penalty")
24:         number(self.wet_penalty, "Wet penalty")

```

Code explanation. Configuration validation keeps penalties finite and restricts the pit factor and seed.

Extract provenance. This is the current-source version, with line numbers, not a claim that every later extension existed at this earlier stage. The preserved run metadata identifies the source hashes used for each actual test run.

Stakeholder review. [GENUINE STAKEHOLDER REVIEW REQUIRED]

Candidate reflection. [CANDIDATE REVIEW AND EXPLANATION REQUIRED]

3.10 Seeded variation

Objective and links. I11: Seeded variation. Requirements FR11; success criteria SC11. Design: docs/design/I11-randomness.md.

Implementation and recorded review. The following preserved iteration account records the implementation, expected and actual results, alternatives and next step. Its claims are bounded by the linked execution evidence.

FR11 design docs/design/I11-randomness.md. Added local RNG, validated seed/amplitude, recorded signed lap variation and conditions-aware comparison. TEST-I11: 21 focused tests passed. Evidence evidence/test-runs/TEST-I11-20260908T211342964557Z/. Exact repeats, changed seeds, bounded noise and untouched global RNG all passed; original dry oracle passed.

No observed failure; remedy/retest N/A. Screenshot pending GUI. [GENUINE STAKEHOLDER FEEDBACK PENDING]

Review: common additive lap noise changes totals but cancels in paired ranking. Monte Carlo must model a strategy-dependent uncertainty such as safety-car timing to change that ranking. No design deviation. Candidate reflection pending; next repeated simulation/statistics.

Code extract: `src/pitwall/engine.py`, lines 9-24 (selected opening simulate)

```

9: def simulate(config: RaceConfig, strategy: Strategy, conditions: Conditions | None = None) ->
StrategyResult:
10:     strategy.validate_for(config)
11:     conditions = conditions or Conditions()
12:     if conditions.variation >= config.circuit.base_lap_time:
13:         raise ValueError("Variation amplitude must be below base lap time")
14:     rng = random.Random(conditions.seed)
15:     queue = EventQueue(conditions.events, config.laps)
16:     weather = conditions.initial_weather
17:     safety_car = False
18:     stops = {stop.lap: stop for stop in strategy.planned_stops}
19:     state = RaceState(0, strategy.starting_compound, 0, 0.0)
20:     results = []
21:     for lap in range(1, config.laps + 1):
22:         for event in queue.at_lap(lap):
23:             if event.kind == EventType.WEATHER_CHANGE:
24:                 weather = event.weather

```


Code explanation. Validation occurs before iteration. State and stop lookup are local to each run; events are processed at the start of a lap.

Extract provenance. This is the current-source version, with line numbers, not a claim that every later extension existed at this earlier stage. The preserved run metadata identifies the source hashes used for each actual test run.

Stakeholder review. [GENUINE STAKEHOLDER REVIEW REQUIRED]

Candidate reflection. [CANDIDATE REVIEW AND EXPLANATION REQUIRED]

3.11 Monte Carlo and statistics

Objective and links. I12: Monte Carlo and statistics. Requirements FR11; success criteria SC11. Design: docs/design/I12-monte-carlo.md.

Implementation and recorded review. The following preserved iteration account records the implementation, expected and actual results, alternatives and next step. Its claims are bounded by the linked execution evidence.

FR11/A12/A13 design docs/design/I12-monte-carlo.md. Added paired runner, trial records and population summaries. TEST-I12: 10 focused tests passed; evidence evidence/test-runs/TEST-I12-20260908T211512627796Z/. Includes actual 100-run zero-noise case, exact seeded event replay, ties, independent statistics oracle and cancellation.

No observed failures; remedy/retest N/A. Screenshot pending integration. [GENUINE STAKEHOLDER FEEDBACK PENDING]

Review: trial records support replay and common conditions support fair pairing. Generated and fixed SC overlap remains a documented modelling limitation. Rain timing is not randomised. Counts 500/1000 are implemented but performance remains to be measured before recommending larger work. No design deviation; candidate reflection pending. Next constrained search.

Code extract: src/pitwall/monte_carlo.py, lines 15-21 (complete describe)

```
15: def describe(values: tuple[float, ...]) -> dict[str, float]:
16:     if not values:
17:         raise ValueError("Statistics require at least one sample")
18:     for value in values:
19:         number(value, "Sample")
20:     return {"mean": statistics.mean(values), "median": statistics.median(values),
21:           "stdev": statistics.pstdev(values), "minimum": min(values), "maximum": max(values)}
```

Code explanation. Summary statistics are calculated from actual totals. Population standard deviation describes the supplied trial population.

Extract provenance. This is the current-source version, with line numbers, not a claim that every later extension existed at this earlier stage. The preserved run metadata identifies the source hashes used for each actual test run.

Stakeholder review. [GENUINE STAKEHOLDER REVIEW REQUIRED]

Candidate reflection. [CANDIDATE REVIEW AND EXPLANATION REQUIRED]

3.12 Bounded optimisation

Objective and links. I13: Bounded optimisation. Requirements FR12; success criteria SC12. Design: docs/design/I13-optimiser.md.

Implementation and recorded review. The following preserved iteration account records the implementation, expected and actual results, alternatives and next step. Its claims are bounded by the linked execution evidence.

FR12/A10/A11 design docs/design/I13-optimiser.md. Added generator, complete capped scoring, cancellation and baseline rules. TEST-I13: 3 tests passed, evidence evidence/test-runs/TEST-I13-20260908T211637569849Z/. Manual 3-lap/one-stop space contained 21 unique strategies; 2-lap optimum was Soft no-stop 182.2 s among 12. Minimum stints and early cap/cancel rejection passed.

No observed failure; remedy/retest N/A. Screenshot pending GUI. [GENUINE STAKEHOLDER FEEDBACK PENDING]

Review: explicit enumeration is defensible, but grid step excludes off-grid plans. Baselines are separate references and may be outside grid. Search suppresses random variation and does not optimise Monte Carlo risk. No claim of global real-world optimality. Candidate reflection pending. Next: persistence before final chart/GUI integration; Iteration14 chart additions will be grouped with 16 because they share the same UI changes.

Code extract: src/pitwall/optimiser.py, lines 32-45 (complete candidates)

```
32: def candidates(config: RaceConfig, settings: SearchSettings):
33:     if settings.minimum_stint > config.laps:
34:         raise ValueError("Minimum stint cannot exceed race length")
35:     grid = range(settings.minimum_stint, config.laps - settings.minimum_stint + 1,
settings.pit_step)
36:     serial = 0
37:     for count in range(settings.max_stops + 1):
38:         for stops in combinations(grid, count):
39:             bounds = (0, *stops, config.laps)
40:             if any(b-a < settings.minimum_stint for a,b in zip(bounds, bounds[1:])):
41:                 continue
42:             for compounds in product(Compound, repeat=count + 1):
43:                 serial += 1
44:                 yield Strategy(f"Candidate {serial}", compounds[0], tuple(
45:                     PitStopPlan(lap, tyre, settings.stationary_time) for lap,tyre in zip(stops,
compounds[1:]))))
```

Code explanation. Combinations choose ordered distinct stop laps; product enumerates compound sequences. Minimum-stint checks remove illegal combinations.

Extract provenance. This is the current-source version, with line numbers, not a claim that every later extension existed at this earlier stage. The preserved run metadata identifies the source hashes used for each actual test run.

Stakeholder review. [GENUINE STAKEHOLDER REVIEW REQUIRED]

Candidate reflection. [CANDIDATE REVIEW AND EXPLANATION REQUIRED]

3.13 JSON persistence

Objective and links. I15: JSON persistence. Requirements FR09; success criteria SC09. Design: docs/design/I15-persistence.md.

Implementation and recorded review. The following preserved iteration account records the implementation, expected and actual results, alternatives and next step. Its claims are bounded by the linked execution evidence.

FR09/A14 design docs/design/I15-persistence.md. Added strict version1 schema and atomic-save module. TEST-I15:7 tests passed, evidence evidence/test-runs/TEST-I15-20260908T211824033057Z/. Exact event/seed/stop-time round-trip passed; malformed JSON, duplicate keys, unsupported versions, invalid length and oversized files rejected. No observed failure; remedy/retest N/A. Screenshot pending GUI. [GENUINE STAKEHOLDER FEEDBACK PENDING]

Review: validating before replacement prevents invalid saved scenarios and temporary files are cleaned. Atomic replacement depends on filesystem semantics. Search/run preferences are separate from race data. Chart iteration14 is intentionally combined with 16 as recorded before implementation. Candidate reflection pending.

Code extract: src/pitwall/persistence.py, lines 117-132 (complete save_scenario)

```
117: def save_scenario(path: Path, scenario: Scenario) -> None:
118:     # Round-trip validation verifies the serialized contract before touching destination.
119:     text = json.dumps(to_data(scenario), default=enum_json, allow_nan=False, indent=2)
120:     from_data(json.loads(text))
121:     if len(text.encode('utf-8')) > MAX_FILE_BYTES:
122:         raise ValueError("Scenario file exceeds 1 MiB")
123:     path = Path(path)
124:     temporary = None
125:     try:
126:         with tempfile.NamedTemporaryFile('w', encoding='utf-8', dir=path.parent, suffix='.tmp',
delete=False) as stream:
127:             temporary = Path(stream.name)
128:             stream.write(text)
129:             os.replace(temporary, path)
130:     finally:
```

```

131:         if temporary is not None and temporary.exists():
132:             temporary.unlink()

```

Code explanation. The serialised contract is validated before a temporary sibling file replaces the destination.

Extract provenance. This is the current-source version, with line numbers, not a claim that every later extension existed at this earlier stage. The preserved run metadata identifies the source hashes used for each actual test run.

Stakeholder review. [GENUINE STAKEHOLDER REVIEW REQUIRED]

Candidate reflection. [CANDIDATE REVIEW AND EXPLANATION REQUIRED]

3.14 Advanced GUI and charts

Objective and links. I14-I16: Advanced GUI and charts. Requirements FR09 FR11 FR12; success criteria SC09 SC11 SC12 SC14. Design: docs/design/I14-I16-interface.md.

Implementation and recorded review. The following preserved iteration account records the implementation, expected and actual results, alternatives and next step. Its claims are bounded by the linked execution evidence.

FR09..FR12 design docs/design/I14-I16-interface.md. Added AdvancedWindow and QThread worker. Input snapshots/revisions protect against stale results, cancellation checked inside algorithms. JSON load checks representable ranges/precision before touching inputs. Additional lap columns reveal weather, safety-car and noise terms. Actual result charts show ranked totals and paired trial distributions.

TEST-I16:79 accumulated tests passed, evidence evidence/test-runs/TEST-I16-20260908T212235136707Z/. Includes a real asynchronous39-candidate search and 100 paired trials. No observed test failures; remedy/retest N/A. Advanced visual capture still pending at this record. [GENUINE STAKEHOLDER FEEDBACK PENDING]

Review: desktop remains responsive during long work; only GUI thread handles widgets. Changed input results are discarded, so user must rerun. Dedicated stop/event editors could improve discoverability over text syntax. Per-stop service override was added to preserve loaded data. Candidate reflection pending; next independent robustness and performance checks.

Code extract: src/pitwall/advanced_gui.py, lines 208-223 (selected opening accept_experiment)

```

208:     def accept_experiment(self, kind, scenario, controls, revision, result):
209:         if revision != self.revision:
210:             self.status.setText("Run completed for older inputs. Result discarded; run again
for current inputs.")
211:             return
212:             self.experiment_figure.clear()
213:             ax = self.experiment_figure.add_subplot(111)
214:             ax.set_facecolor('#111d2c')
215:             ax.tick_params(colors='#bbccdf')
216:             if kind == "search":
217:                 scores, refs = result
218:                 best = scores[0]
219:                 self.headline.setText(f"Search complete | {len(scores):,} candidates | Best
{best.total_time:,.3f} s")
220:                 plan = best.strategy
221:                 stops = ', '.join(f"{p.lap}:{p.compound.value}" for p in plan.planned_stops) or
'none'
222:                 text = f"Examined {len(scores)} legal candidates.\nBest within this search:
{best.total_time:,.3f} s\nStart: {plan.starting_compound.value}; stops after lap: {stops}\nVariation
disabled; fixed events retained.\n\nSeparate baseline references:\n"
223:                 text += '\n'.join(f"{r.strategy.name}: {r.total_time:,.3f} s (gap to best
{r.total_time-best.total_time:+.3f} s)" for r in refs)

```

Code explanation. The revision guard discards results for changed inputs before charts or output text are populated.

Extract provenance. This is the current-source version, with line numbers, not a claim that every later extension existed at this earlier stage. The preserved run metadata identifies the source hashes used for each actual test run.

Figure 4 shows the number of legal candidates and baseline reference totals produced by the actual run.

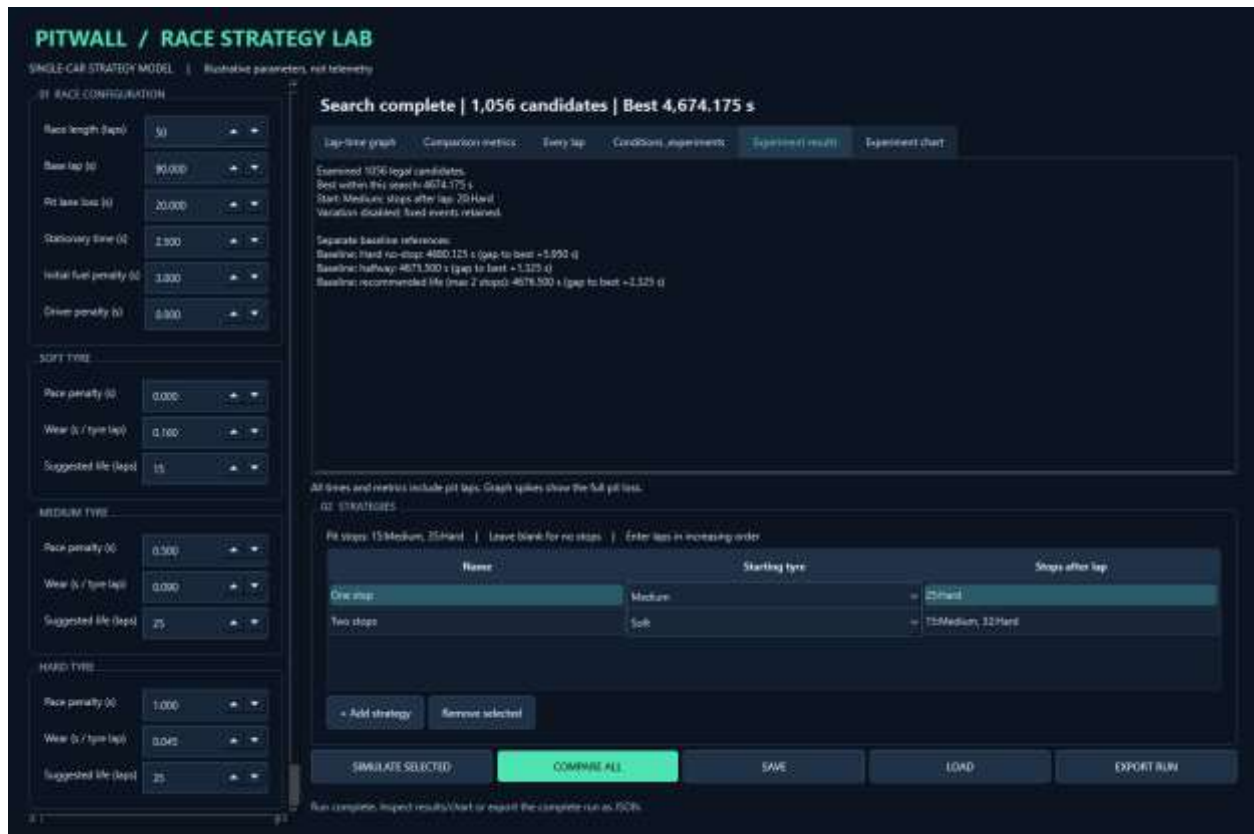


Figure 4: Completed bounded-search result

Stakeholder review. [GENUINE STAKEHOLDER REVIEW REQUIRED]

Candidate reflection. [CANDIDATE REVIEW AND EXPLANATION REQUIRED]

3.15 Robustness and verification

Objective and links. I17: Robustness and verification. Requirements FR04 FR09; success criteria SC04 SC09 SC13 SC14. Design: docs/design/I17-verification.md.

Implementation and recorded review. The following preserved iteration account records the implementation, expected and actual results, alternatives and next step. Its claims are bounded by the linked execution evidence.

Objective: I17 plan in docs/design/I17-verification.md; reinforce FR04/FR09 and evaluate FR11/FR12. New independent checks cover JSON container types, full tiny search arithmetic, reconstructing Monte Carlo trials and maximum comparison.

Genuine failure: TEST-I17-ROBUSTNESS had2 failed/3 passed. Empty objects were incorrectly accepted as event/stop arrays. Raw evidence was saved and committed before adding explicit array validation. Diagnosis, alternatives, selected remedy and unchanged-case retest: docs/testing/FAIL-I17-01.md. Retest84 passed.

A real capture harness timing failure and remedy are documented in docs/testing/FAIL-I17-02.md; partial captures remain. Complete captures subsequently succeeded. Visual review led to clearer completed-experiment headings and shared histogram bins, without changing underlying simulation data.

Final execution TEST-FINAL:86 passed in 6.14s. Evidence evidence/test-runs/TEST-FINAL-20260908T213416753101Z/. Includes new GUI stale-result and cancellation tests. This is actual automated output, not proof of genuine user usability.

Final actual Windows captures: evidence/screenshots/FIG-I16-20260908T213427339706Z/, eight PNG views plus full1056-candidate search and 100-trial Monte Carlo JSON. The earlier first-milestone capture is preserved separately. BENCH-01 raw repeated timing and actual machine metadata are in evidence/benchmarks/BENCH-01-20260908T212651988284Z/.

What worked: deterministic oracle remained correct through environmental extensions; tiny independent enumeration matched search; explicit JSON validation fixed a real gap. Difficulty: deep Windows paths prevented initial PySide6 installation, and asynchronous result delivery required the capture harness to wait for data rather than thread completion alone. Alternative: a dedicated GUI testing library could provide signal waits, but current harness keeps dependencies small.

Lesson: validate containers as well as their elements, and distinguish computation completion from GUI signal delivery. Comparison with design: planned combined stages and persistence order change recorded before implementation; optional wet tyres remain deferred. [GENUINE STAKEHOLDER FEEDBACK PENDING]

Candidate personal reflection remains pending. Next work should be genuine stakeholder/candidate/centre review, targeted changes arising from it, and preparing the final assessed document. Do not manufacture those inputs or call the mark audit complete.

Code extract: `src/pitwall/persistence.py`, lines 53-56 (complete array)

```
53: def array(data, name):
54:     if not isinstance(data, list):
55:         raise ValueError(f"{name} must be a JSON array")
56:     return data
```

Code explanation. The type guard rejects objects even when empty, closing the exact reproduced decoder gap.

Extract provenance. This is the current-source version, with line numbers, not a claim that every later extension existed at this earlier stage. The preserved run metadata identifies the source hashes used for each actual test run.

Stakeholder review. [GENUINE STAKEHOLDER REVIEW REQUIRED]

Candidate reflection. [CANDIDATE REVIEW AND EXPLANATION REQUIRED]

3.16 Genuine failures and remedial action

Observed during actual final robustness execution on 2026-09-08. Evidence saved BEFORE remedial changes: `evidence/test-runs/TEST-I17-ROBUSTNESS-20260908T212523289799Z/` (2 failed, 3 passed; output, JUnit, source hashes).

Input: replace `conditions.events` or a strategy's `planned_stops` JSON array with `{}`. Expected: `ValueError` for incorrect collection type. Actual: loader iterated zero dictionary keys and accepted an empty tuple. This is a genuine decoder bug, not an intentionally introduced failure. It was identified by review and reproduced through new adversarial tests.

Root cause: exact dictionary field checks validated each element but did not validate the container type before iteration. An empty invalid container therefore never reached element checks.

Options: (1) explicit JSON-array guard at each collection boundary; (2) introduce a schema-validation dependency; (3) accept any iterable. Selected 1 because the schema is small, keeps dependencies minimal and rejects data outside the documented contract. Option 3 would preserve the bug's ambiguity.

Planned remedy: helper requiring list with explicit name, used for `tyres`, `events`, `strategies`, `planned_stops`. Planned retest: unchanged failing cases plus complete regression. Design impact: make collection validation explicit before element validation. Retest results will be added after execution.

Executed remedy and retest: added `array()` before all four collection loops. TEST-I17-RETEST executed with 84 passed, including both unchanged reproduction cases. Evidence: `evidence/test-runs/TEST-I17-RETEST-20260908T212641586436Z/`. Original failed output remains unchanged. Full regression did not show additional failures.

Actual scripts/capture_advanced.py execution raised `RuntimeError: Running. You can cancel; editing inputs discards this run's displayed result.` at line 61. The script checked `worker.isRunning()` and assumed the queued result had already arrived. It had not. Initial four actual screenshots and capture metadata remain in the first FIG-I16 directory. Tool output remains in this task; this record transcribes the actual message rather than inventing a test result.

The failed component is the capture harness. A completed thread does not guarantee its queued GUI signal has been processed. The earlier automated integration test explicitly processed events after thread completion; this script lacked that synchronisation.

Remedy: wait for the actual experiment_data result before capturing; keep the existing bounded timeout. Add a guarded callback that exits with an error and cancels/joins a worker on exceptions, avoiding a stranded capture window. Do not change simulation output or render fabricated result text. Retest must produce actual complete captures and run JSON.

Executed retest: process exited0; eight actual views and complete search/Monte Carlo JSON were captured in evidence/screenshots/FIG-I16-20260908T213008377764Z/. Visual inspection covered the comparison, controls, Monte Carlo results and histogram. The first failed capture directory FIG-I16-20260908T212913236182Z is retained. The original stuck capture process was interrupted before rerun.

Verbatim lines from evidence/test-runs/TEST-I17-ROBUSTNESS-20260908T212523289799Z/output.txt

```
tests/test_final_robustness.py::test_json_collection_types[events] FAILED [ 20%]
tests/test_final_robustness.py::test_json_collection_types[planned_stops] FAILED [ 40%]
E       Failed: DID NOT RAISE ValueError
E       Failed: DID NOT RAISE ValueError
FAILED tests/test_final_robustness.py::test_json_collection_types[events] - F...
FAILED tests/test_final_robustness.py::test_json_collection_types[planned_stops]
===== 2 failed, 3 passed in 0.59s =====
```

The two failing assertions expected ValueError when {} replaced an events or planned_stops array. Iterating an empty dictionary produced no element checks. The selected remedy validates the container before its contents. A schema dependency was an alternative, but an explicit list check fits this small schema. The unchanged reproduction tests passed in the 84-test retest and the final 86-test run. The capture-harness failure is separately supported by a factual transcription and retained partial output, not a fabricated pytest screenshot.

3.17 Recorded test progression

Table 20: All preserved execution runs

Run ID	Cases	Failures
TEST-FINAL-20260908T213416753101Z	86	0
TEST-I01-20260908T205847968425Z	20	0
TEST-I02-20260908T205945777891Z	27	0
TEST-I03-20260908T210046769795Z	33	0
TEST-I05-20260908T210214046535Z	41	0
TEST-I06-20260908T210320581715Z	44	0
TEST-I07-20260908T210702638724Z	48	0
TEST-I08-20260908T210944391018Z	2	0
TEST-I09-20260908T211102758207Z	12	0
TEST-I10-20260908T211214733077Z	16	0
TEST-I11-20260908T211342964557Z	21	0
TEST-I12-20260908T211512627796Z	10	0
TEST-I13-20260908T211637569849Z	3	0
TEST-I15-20260908T211824033057Z	7	0
TEST-I16-20260908T212235136707Z	79	0

Run ID	Cases	Failures
TEST-I17-RETEST-20260908T212641586436Z	84	0
TEST-I17-ROBUSTNESS-20260908T212523289799Z	5	2
TEST-QUALITY-FINAL-20260908T224148466396Z	87	0
TEST-SC05-20260908T224145702053Z	1	0

Repeated regression tests occur in several runs; these counts must not be added and described as unique tests. Appendix B lists the final 87 collected cases, including parameterised cases. Source hashes and exact commands remain in each run directory.

4 Post-development testing

4.1 Functional testing

The final recorded regression contains 87 passing cases. The rows below reference actual tests and assertions rather than invented manual sessions. Test functions may assert several related conditions. Native file-dialog interaction has not been manually tested; the persistence functions and widget conversion are covered.

Table 21: Functional verification

Test ID	Success criterion	Feature	Input / steps	Expected	Actual	Pass / fail	Evidence
F01	SC05 SC06	Race	5 laps; Soft to Hard after 2	92,114.2,92,91.55,91.1; total 480.85	Assertions matched expected values.	PASS	Appendix B, case 015
F02	SC07	Compare	No-stop Soft versus pit plan	Soft faster by 23.85; average 91.4	Assertions matched expected values.	PASS	Appendix B, case 006
F03	SC08	GUI	Click compare; edit laps	2 lines,50 points each; results clear	Assertions matched expected values.	PASS	Appendix B, case 031
F04	SC09	JSON	Save/load seed,events,service time	Exact Scenario equality	Assertions matched expected values.	PASS	Appendix B, case 060
F05	SC10	SC event	SC laps 2-3; stop after 2	Pit11.25; total 519.6	Assertions matched expected values.	PASS	Appendix B, case 011
F06	SC11	Monte Carlo	100 fixed trials; seeded replay	Repeatable totals and ties	Assertions matched expected values.	PASS	Appendix B, case 055
F07	SC12	Search	4-lap tiny enumeration	Complete scores match direct arithmetic	Assertions matched expected values.	PASS	Appendix B, case 027
F08	SC14	Cancel	Start1000 trials and cancel	No completed output; cancelled status	Assertions matched expected values.	PASS	Appendix B, case 005

All entries above are supported by evidence/test-runs/TEST-QUALITY-FINAL-20260908T224148466396Z/results.xml. Appendix B maps the exact test inventory; assertions reside in tests/test_engine.py, test_comparison.py, test_gui.py, test_persistence.py, test_conditions.py, test_monte_carlo.py, test_final_robustness.py and test_advanced_gui.py.

4.2 Robustness testing

Table 22: Recorded invalid and boundary cases

Area	Input	Expected and actual	Evidence test
Race	0,-1,501,2.5,True	ValueError; PASS	test_invalid_race_length
Stops	Duplicate,unordered,final lap	Rejected; PASS	test_stop_order_and_race_end
Events	Unknown command or invalid lap	Rejected; PASS	test_events_parser / test_bad_events

Area	Input	Expected and actual	Evidence test
JSON	Malformed,missing,duplicates,NaN	Rejected; PASS	test_malformed
Schema	2,True,string1; oversized file	Rejected; PASS	test_schema_and_model_validation
Containers	{ } in events/stops	Initially accepted; fixed and PASS	test_json_collection_types
Monte Carlo	0,-1,1001,True	Rejected; PASS	test_invalid_counts_and_cancel
Search	Lowered cap; cancellation callback	Explicit error/interruption; PASS	test_limit_and_cancel
Large comparison	20 strategies x500 laps	20 equal complete results; PASS	test_maximum_comparison

4.3 Performance testing

BENCH-01 records three repeats after one unmeasured single-race warm-up. Each race has 50 laps. Timings use perf_counter and include computational result construction, not GUI rendering. The CPU is Snapdragon(R) X 10-core X1P64100 @ 3.40 GHz. Physical RAM is 16756006912 bytes. Python is 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)].

Table 23: Raw measured seconds and summary

Operation / count	Trial 1	Trial 2	Trial 3	Min	Max	Mean
sequential races / 1	0.000532	0.000524	0.000511	0.000511	0.000532	0.000522
sequential races / 100	0.061353	0.075801	0.065233	0.061353	0.075801	0.067462
sequential races / 500	0.341047	0.319111	0.334610	0.319111	0.341047	0.331589
sequential races / 1000	0.807915	0.670294	0.910702	0.670294	0.910702	0.796304
candidate prefix scoring (not full search) / 10	0.006008	0.009976	0.009137	0.006008	0.009976	0.008374
candidate prefix scoring (not full search) / 100	0.083678	0.086033	0.087804	0.083678	0.087804	0.085839
candidate prefix scoring (not full search) / 1000	0.892307	1.054861	1.032549	0.892307	1.054861	0.993239
complete default search / 1056	1.350378	1.029638	1.104884	1.029638	1.350378	1.161633
paired Monte Carlo trials / 100	0.206715	0.226102	0.217682	0.206715	0.226102	0.216833
paired Monte Carlo trials / 500	1.113949	1.136409	1.229209	1.113949	1.229209	1.159856
paired Monte Carlo trials / 1000	2.516607	1.691347	2.089219	1.691347	2.516607	2.099058

The complete default search examined 1,056 candidates and averaged 1.161633 seconds. The 1,000 paired-trial Monte Carlo run averaged 2.099058 seconds. Both are below the recorded 10-second and 30-second targets on this machine. Prefix scoring benchmarks measure throughput on the first candidates, not complete optimisation. Background load and power state were not controlled, so results cannot establish minimum hardware or another device's latency. Appendix D retains the raw JSON precision.

4.4 Manual verification

Five laps, base 90, initial fuel penalty 2, lane loss 20, stationary 2.5. Soft pace 0/wear .2; Hard pace 1/wear .05. Stop after lap 2.

Table 24: Five-lap independent expected calculation

Lap	Racing compound	Age	Wear	Fuel	Pit	Lap time	Cumulative
1	Soft	0	0	2	0	92	92
2	Soft	1	.2	1.5	22.5	114.2	206.2
3	Hard	0	0	1	0	92	298.2
4	Hard	1	.05	.5	0	91.55	389.75
5	Hard	2	.1	0	0	91.1	480.85

These are hand-derived expected values, not executed output. No-stop Soft expected 92, 91.7, 91.4, 91.1, 90.8 = 457.0 seconds.

Planned tests: manual oracle; repeated identical runs; immutable config; zero-stop one-lap race; first and penultimate lap stops; consecutive stops with same compound; invalid final stop; zero pit loss; 500-lap output count; total equals sum. Extreme finite inputs that overflow must fail descriptively rather than return infinity.

For lap 2, 90 base +0 Soft pace +0.2 wear +1.5 fuel +22.5 pit =114.2 seconds. The new Hard set starts lap 3 at age 0:90+1+0+1=92. The sum is 480.85. With a safety car on laps 2 and 3, the model adds 50 seconds but saves 11.25 seconds on the pit loss, giving 519.6. These are manual expected calculations matched by executed assertions, not a claim of a separate human testing session.

For statistics [1,2,3], mean and median are 2. Squared deviations are 1,0,1; population variance is 2/3 and standard deviation is $\sqrt{2/3}$. The optimiser's three-lap, maximum-one-stop space contains 3 no-stop plans plus 2 legal stop laps times 9 compound pairs, giving 21. The independently calculated four-lap score distribution is also tested.

4.5 Usability testing

[USABILITY TESTING MUST BE COMPLETED WITH GENUINE USERS]

Status: PLANNED. No participant has performed this protocol.

Participant identifier: [GENUINE PARTICIPANT REQUIRED]

Date/time and consent to record comments: [PENDING]

Device/display scaling: [PENDING]

Table 25: Unconducted usability task sheet

Task	Completion	Mistakes/confusion	Help required	Verbatim comment
Open and configure a five-lap race	Pending	Pending	Pending	[GENUINE RESPONSE REQUIRED]
Create two strategies and add a legal stop	Pending	Pending	Pending	[GENUINE RESPONSE REQUIRED]
Compare and explain the pit spike	Pending	Pending	Pending	[GENUINE RESPONSE REQUIRED]
Correct an invalid final-lap stop	Pending	Pending	Pending	[GENUINE RESPONSE REQUIRED]

Task	Completion	Mistakes/confusion	Help required	Verbatim comment
Apply an event and explain before/after-lap distinction	Pending	Pending	Pending	[GENUINE RESPONSE REQUIRED]
Run/cancel search and explain its scope	Pending	Pending	Pending	[GENUINE RESPONSE REQUIRED]
Run Monte Carlo and interpret probability	Pending	Pending	Pending	[GENUINE RESPONSE REQUIRED]
Save/load scenario and export a run	Pending	Pending	Pending	[GENUINE RESPONSE REQUIRED]

Observe keyboard navigation, legibility, status feedback and whether participants assume model outputs are real predictions. Record actual duration if timed; do not estimate retrospective timings. Ask which input or label they would change. Record resulting requirement changes with links to the original response.

Use neutral instructions and record actual help, mistakes, completion and comments. Ask the participant to interpret a pit spike and explain what an 88% simulated win frequency means. Do not imply it forecasts a real race. Appendix F includes blank response space and stakeholder follow-up.

4.6 Final quality verification

SC05 focused run: evidence/test-runs/TEST-SC05-20260908T224145702053Z. One test passed. Absolute tolerance is 1e-9 seconds with no relative component. Every recorded lap and cumulative value had zero difference from its expected Python float value; this is not a claim of infinite decimal precision. The test also checks total time and deterministic replay.

Full regression: evidence/test-runs/TEST-QUALITY-FINAL-20260908T224148466396Z. All 87 collected cases passed. The old 86-case run and both I17 failing assertions remain preserved.

Comparison-only measurements: evidence/benchmarks/BENCH-COMPARE-20260908T224158666794Z/timings.json. Five repeats after one warm-up per size. Input construction and GUI drawing are excluded. No new speed threshold was invented. Raw output, environment, command and source hashes are retained.

Table 26: Comparison latency summary

Strategies	Laps	Minimum s	Maximum s	Mean s	Median s
2	50	0.000999200	0.001561200	0.001120600	0.001018200
20	50	0.010145800	0.014767600	0.011749560	0.011352200
20	500	0.153045700	0.179047800	0.161278460	0.157290200

SC05 and the missing SC13 computational evidence are now satisfied for the tested cases. SC04 and SC14 still need genuine human evidence. No application source was changed and no new feedback-driven iteration was fabricated.

Table 27: Raw comparison repeats in seconds

Strategies / laps	Repeat 1	Repeat 2	Repeat 3	Repeat 4	Repeat 5
2 / 50	0.001561200	0.001018200	0.001021400	0.001003000	0.000999200
20 / 50	0.010145800	0.011352200	0.010986400	0.011495800	0.014767600
20 / 500	0.157290200	0.179047800	0.153045700	0.155666100	0.161342500

Only compare() is timed. Each input uses default circuit, tyre and fuel settings, rotating dry starting compounds and one Hard stop at halfway. No events or random variation are enabled. Five repetitions and one warm-up per size show local variability; they do not measure human responsiveness or establish a new acceptance threshold.

Executed explicit SC05 tolerance test

```

"""SC05: absolute error at most 1e-9 seconds, with no relative tolerance."""
from pitwall.engine import simulate
from pitwall.models import Compound, PitStopPlan, Strategy

def test_five_lap_oracle_absolute_tolerance(config):
    tolerance_seconds = 1e-9
    strategy = Strategy("SC05 oracle", Compound.SOFT, (PitStopPlan(2, Compound.HARD),))
    result = simulate(config, strategy)
    expected_laps = [92.0, 114.2, 92.0, 91.55, 91.1]
    expected_cumulative = [92.0, 206.2, 298.2, 389.75, 480.85]
    assert len(result.laps) == len(expected_laps) == 5
    errors = []
    for lap, expected, cumulative in zip(result.laps, expected_laps, expected_cumulative):
        lap_error = abs(lap.lap_time - expected)
        cumulative_error = abs(lap.cumulative_time - cumulative)
        print(f"lap={lap.lap_number} actual={lap.lap_time:.17g} expected={expected:.17g} "
              f"absolute_error={lap_error:.17g} cumulative_error={cumulative_error:.17g}")
        assert lap_error <= tolerance_seconds
        assert cumulative_error <= tolerance_seconds
        errors.extend((lap_error, cumulative_error))
    assert abs(result.total_time - 480.85) <= tolerance_seconds
    assert simulate(config, strategy) == result
    print(f"SC05 absolute tolerance={tolerance_seconds:.17g} s; maximum error={max(errors):.17g} s")

```

Recorded SC05 differences

```

tests/test_sc05_tolerance.py::test_five_lap_oracle_absolute_tolerance lap=1 actual=92 expected=92
absolute_error=0 cumulative_error=0
lap=2 actual=114.2 expected=114.2 absolute_error=0 cumulative_error=0
lap=3 actual=92 expected=92 absolute_error=0 cumulative_error=0
lap=4 actual=91.54999999999997 expected=91.54999999999997 absolute_error=0 cumulative_error=0
lap=5 actual=91.09999999999994 expected=91.09999999999994 absolute_error=0 cumulative_error=0
SC05 absolute tolerance=1.0000000000000001e-09 s; maximum error=0 s

```

The focused run is evidence/test-runs/TEST-SC05-20260908T224145702053Z. The full run is evidence/test-runs/TEST-QUALITY-FINAL-20260908T224148466396Z/results.xml. Both retain output, XML, command, environment and source hashes. Original test and failure records remain unchanged. This is verification of the existing program, not a new stakeholder-led software iteration.

5 Evaluation

5.1 Overall evaluation

PITWALL solves the computational comparison problem within its simplified assumptions. It accepts defined strategies, calculates each lap, ranks outcomes and explores bounded choices and uncertainty. The 87 recorded passing tests, preserved failure/fix sequence, actual GUI captures and benchmarks support these functions. The wider success claim remains conditional because genuine stakeholder needs and usability have not yet been evaluated.

5.2 Evaluation against success criteria

Table 28: Success criterion judgements

ID	Criterion	Evidence	Judgement	Scope / reason	Remaining work
SC01	Accept integer race lengths 1..500 and positive base lap time; reject 0, negative, fractional and oversized lengths	Race boundaries/invalid types in test_models and maximum comparison in test_final_robustness	FULLY MET computationally	Illustrative model; human acceptance pending.	Candidate review and target-user acceptance.
SC02	All three compounds have configurable non-negative pace penalties and linear degradation; age 3 at 0.2 s/lap gives 0.6 s	Linear wear manual examples and editable three-compound inputs	FULLY MET computationally	Illustrative model; human acceptance pending.	Candidate review and target-user acceptance.
SC03	Ordered distinct stops after laps 1..N-1 change compound on the following lap and add pit loss exactly once	Pit indexing, duplicate/final stops, consecutive same-compound resets tested	FULLY MET computationally	Illustrative model; human acceptance pending.	Candidate review and target-user acceptance.
SC04	Automated typical, boundary and erroneous cases raise descriptive errors; GUI shows an error for malformed stops	Model/parser errors and GUI recovery tested; genuine users have not judged explanations	PARTIALLY MET	Illustrative model; human acceptance pending.	Run genuine error-message usability tasks.
SC05	Two runs of identical inputs return equal results; five-lap manual oracle matches each lap within 1e-9 seconds	Explicit absolute 1e-9-second oracle passes for every lap, cumulative time and total; exact replay passes. See docs/testing/final-quality-results.md	FULLY MET	Illustrative model; human acceptance pending.	Preserve the explicit tolerance regression.
SC06	Exactly N lap rows; total equals sum of lap times; tyre age resets after a stop	Lap count/sum checks and actual Qt table capture	FULLY MET computationally/visually	Illustrative model; human acceptance pending.	Candidate review and target-user acceptance.
SC07	At least two strategies produce ranked total, average, fastest, slowest, pit loss, stop count and stint lengths	Rankings, ties, metrics and stints tested; real-world strategy validity not claimed	FULLY MET in model	Illustrative model; human acceptance pending.	Candidate review and target-user acceptance.
SC08	Real Qt application opens and renders a graph with N points per compared strategy; manual usability remains separately pending	Actual Windows captures and graph-point tests; audience usability pending	FULLY MET for rendering	Illustrative model; human acceptance pending.	Obtain audience interpretation evidence.

ID	Criterion	Evidence	Judgement	Scope / reason	Remaining work
SC09	Exact scenario round-trip with seed/events/stop service; reject wrong schema, malformed data and collection types	Exact round-trip and robust JSON checks; genuine discovered collection bug fixed/retested	FULLY MET for tested schema	Illustrative model; human acceptance pending.	Candidate review and target-user acceptance.
SC10	Events chronological; ties input-ordered; wet increments30s and SC manual result519.6s in documented five-lap examples	Queue tie tests and weather/SC arithmetic; wet tyre selection remains unsupported	FULLY MET for simplified events	Illustrative model; human acceptance pending.	Candidate review and target-user acceptance.
SC11	Same seed reproduces full trials; summary oracle [1,2,3] correct;100 actual paired trials; count0/1001 rejected	Seed replay, fixed statistics oracle, actual trial output and 1000-trial benchmark	FULLY MET for selected uncertainties	Illustrative model; human acceptance pending.	Candidate review and target-user acceptance.
SC12	N3/max1 space contains 21 unique plans; tiny winner182.2s; independent4-lap score distribution matches; report complete candidate count	Independent tiny exhaustive arithmetic, candidate counts and exported complete score list	FULLY MET in constrained grid	Illustrative model; human acceptance pending.	Candidate review and target-user acceptance.
SC13	Recorded machine default search <10s and 1000 paired50-lap trials <30s (three repeats)	Original search/Monte Carlo targets passed; separate five-repeat comparison timings now recorded. GUI latency and other machines remain outside these measurements. See docs/testing/final-quality-results.md	FULLY MET	Illustrative model; human acceptance pending.	Verify GUI latency and other target machines separately.
SC14	Background task returns results; old-input results discarded; cancellation callback honoured; GUI data round-trip exact	Worker tests/cancellation and stale-input checks; genuine manual responsiveness observations pending	PARTIALLY MET	Illustrative model; human acceptance pending.	Record manual responsiveness and cancellation.

SC05 is now FULLY MET for the recorded five-lap example: a new explicit absolute 1e-9-second test checks each lap, cumulative time and total, with exact replay. Section 4.6 records the executed evidence. The original looser test remains intact.

SC13 computational evidence is now complete: separate comparison timings supplement the recorded search and Monte Carlo benchmarks. These measurements exclude GUI rendering. SC04 and SC14 remain PARTIALLY MET because genuine error-interpretation and responsiveness observations are missing.

5.3 Functional correctness

Race, tyre and fuel calculations match the small independent oracle. Stop checks prevent illegal ordering and the engine resets age on the following lap. Event tests establish chronological processing and stable ties. Comparison metrics include pit laps consistently. Monte Carlo reproduces seeds and trial schedules; statistics match known samples. Search matches a separate exhaustive arithmetic calculation in a tiny space. Save/load preserves typed scenarios and rejects malformed schema data after the genuine collection-type fix. These assertions support correctness of tested cases rather than proving the entire program bug-free.

5.4 Evaluation of algorithms

Table 29: Algorithm strengths and trade-offs

Algorithm	Strength	Limitation and alternative
Race loop	$O(N)$ without events; each component inspectable	No within-lap physics; fractional segments add complexity
Event heap	Stable chronological processing, $O(E \log E)$ total removals	Current static input could be sorted once more simply
Paired Monte Carlo	Replayable common conditions; trial audit retained	Uncalibrated distributions; add sensitivity study before richer uncertainty
Bounded enumeration	Every legal candidate in declared grid scored	Off-grid optimum may be missed; coarse-to-fine heuristic needs exact baseline
Strict JSON	Exact keys and model validation reject ambiguous inputs	Schema evolution needs explicit migrations

Monte Carlo does not make the model more realistic merely by repeating it. With common additive noise the strategy gap is unchanged unless another effect interacts with the plan. Random safety-car timing provides that interaction through pit loss. The reported 88% in the captured 100-trial example is an empirical frequency under the selected seed and probability, not a calibrated probability of real success.

5.5 Performance

Measured computational runs complete in seconds at the selected desktop scope. Increasing trial count approximately increases repeated work; increasing pit-lap choices can grow the search much faster because pairs of stop laps and compound sequences multiply. Limits protect the interface from uncontrolled enumeration. Worker cancellation is checked between trials/candidates, not in the middle of each lap. A single very long operation could therefore delay cancellation until its current simulation finishes, though laps are capped.

5.6 Robustness

The empty-object decoder failure demonstrates why testing must include structurally wrong but syntactically valid JSON. Checking each member did not validate the collection itself. The repair is narrow and the reproduction tests remain in the suite. Other checks cover finite numbers, types, boundaries and oversized files. The GUI refuses values it cannot represent at three decimal places rather than silently clamping a loaded scenario. Further testing should include native file-dialog cancellation, failed disk writes and target-machine scaling.

5.7 Usability

[FINAL USABILITY EVALUATION PENDING GENUINE USER TESTING]

Actual captures show labelled controls, graph axes, tabs and output tables. Automated tests check error recovery, invalidation and cancellation. These features appear appropriate for the task, but no recorded participant has demonstrated that the stop/event syntax or experiment controls are easy to understand. The scrollable panel may hide advanced actions below the initial view.

5.8 Stakeholder evaluation

[GENUINE STAKEHOLDER EVALUATION REQUIRED]

Use the questions and forms in Appendix F. Record requirements changed by feedback and link each decision to a genuine response. Candidate review of algorithms and centre approval are separate outstanding inputs.

5.9 Maintenance

Table 30: Maintenance issues and technical changes

Issue	Actual affected architecture	Technical maintenance approach	Risk/test needed
New tyre compounds	Compound enum, RaceConfig completeness, defaults, GUI, persistence	Add explicit compound data and suitability, migrate schema if needed	Test completeness, serialization and mixed-weather behaviour
Changed formulas	physics.py, Conditions terms and engine	Isolate revised formula and retain versioned assumption notes	Recompute independent oracle without simply copying program output
New event types	RaceEvent payload validation, EventQueue, engine branches, parser/schema	Define new payload and deterministic same-lap semantics before implementation	Contradictory/same-lap and boundary-event tests
Schema evolution	persistence.py exact keys and version1	Introduce explicit version2 decoder and migration from1	Golden legacy fixtures, malformed/new field tests, no silent coercion
GUI growth	MainWindow/AdvancedWindow inheritance and snapshot conversion	Extract dedicated configuration/strategy panels while keeping pure model boundary	Round-trip, stale-output and keyboard/usability regressions
Search changes	candidates, SearchSettings, caps and baseline definitions	Compare a new heuristic against tiny exact oracle and measured exhaustive reference	Report search coverage, timing and any missed best candidate

5.10 Limitations

Table 31: Limitations and improvements

Limitation	Consequence	Possible implementation
Linear, uncalibrated degradation	Cannot predict thermal/cliff behaviour	Add optional quadratic/cliff coefficients only after empirical justification, explicit units and oracle tests
No wet/intermediate tyres	All dry compounds get same rain penalty	Extend Compound and per-weather suitability matrix with separate documented defaults and schema migration
Single-car/free-track abstraction	No traffic, overtaking or undercut interaction	Add field state and gap model as a separately evaluated extension; avoid claiming present ranking works in traffic
Lap-boundary events	Cannot resolve a mid-lap shower or SC deployment	Optional fractional-lap segments with explicit timing semantics and increased complexity tests
SC assignment model	Overlapping fixed/generated periods can override each other	Represent SC intervals with union semantics, compare against existing model and update event design
Shared additive noise and arbitrary SC probability	Noise alone cannot change relative pace ranking; probability is not a forecast	Introduce justified strategy-dependent uncertainties only with assumptions, paired draws and sensitivity studies
Grid-limited two-stop search	Better off-grid or three-stop plans may exist	Coarse-to-fine search, explicit evaluated regions and full tiny-space reference tests

Limitation	Consequence	Possible implementation
GUI range/precision bounds and temporary runtime	Some valid API scenarios cannot load into widgets; runtime can be cleaned	Dedicated packaging, adjustable numeric precision and verified installer on target OS

Severity is high for claims about real race predictions: uncalibrated parameters, missing traffic and no wet tyres can change the correct decision. Severity is moderate for educational use: lap-boundary events and a simplified SC model are acceptable if explained. GUI precision and temporary runtime packaging are practical deployment limitations rather than physics errors. Search bounds are intentional and must always accompany the recommendation.

5.11 Future improvements

First collect genuine feedback and validate installation on an intended user's machine. A dedicated stop/event editor can remove syntax mistakes without changing the model. Next, add wet/intermediate compounds only with a documented suitability matrix and schema migration. A non-linear wear model needs a credible calibration dataset and new independent arithmetic tests. A multi-car model or union of overlapping SC intervals should be a separate designed iteration because it changes event semantics. A coarse-to-fine search could reduce work, but must be compared with exact tiny-space results and report which regions were examined.

5.12 Final conclusion

PITWALL demonstrates decomposition, typed data, validation, deterministic state updates, priority queues, repeated simulation and constrained search in a working desktop application. The evidence supports an explainable prototype that compares strategies consistently under its assumptions. Final acceptance requires candidate explanation, genuine user evidence and review against the actual centre checklist. The model remains an educational simulation, not professional race engineering software.

6 Bibliography

References use organisation, title and URL. Publication dates are not invented. Existing-system sources were reviewed after the prototype; technical documentation was checked during report preparation. No source is claimed as an original design influence before the recorded research date.

[R01] TUMFTM. Race simulation repository. <https://github.com/TUMFTM/race-simulation>. Existing-solution review recorded 8 September 2026. Access record: 8 September 2026.

[R02] FastF1 maintainers. FastF1 repository and README. <https://github.com/theOehrly/Fast-F1>. Existing-solution review recorded 8 September 2026. Access record: 8 September 2026.

[R03] Frontier. F1 Manager Belgian Grand Prix guide. <https://www.f1manager.com/nl-NL/2023/grand-prix/belgian-grand-prix>. Publisher guide, existing review. Access record: 8 September 2026.

[R03b] Frontier. F1 Races Brought To Life. <https://www.f1manager.com/features/new/levensechte-f1-races>. Publisher feature article, existing review. Access record: 8 September 2026.

[R04] Python Software Foundation. heapq documentation. <https://docs.python.org/3/library/heapq.html>. Priority queues and stable tie-breaks. Access record: 8 September 2026.

[R05] Python Software Foundation. random documentation. <https://docs.python.org/3/library/random.html>. Local seeded generators. Access record: 8 September 2026.

[R06] The Qt Company. QThread documentation. <https://doc.qt.io/qtforpython-6/PySide6/QtCore/QThread.html>. Worker thread and signals. Access record: 8 September 2026.

[R07] Matplotlib contributors. Backends documentation. <https://matplotlib.org/stable/users/explain/figure/backends.html>. Embedding figures. Access record: 8 September 2026.

[R08] Python Software Foundation. statistics documentation. <https://docs.python.org/3/library/statistics.html>. Mean, median and population standard deviation. Access record: 8 September 2026.

[R09] OCR. H446 specification. <https://www.ocr.org.uk/Images/170844-specification-accredited-a-level-gce-computer-science-h446.pdf>. Content printed pages 13-14 and marking grids 22-25 checked; downloaded content headers 3.0/2026, retained grid headers 2.6/2023. Access record: 8 September 2026.

Primary project sources are the actual files cited throughout: docs/design, docs/iterations, source and test files, test-run XML/text/metadata, benchmark JSON, scenario JSON and screenshot capture records. Git timestamps are reproduced in Appendix G. The assistance declaration is maintained in docs/assistance-log.md and reproduced once in Appendix K.

7 Appendices

Appendix A Source and class relationships

Selected implementation extracts appear in Section 3. The full source remains in `src/pitwall` and is hash-indexed; repeating every GUI line would add bulk without improving the main argument. The following UML-style class relationships and complete core formulas supplement those extracts.

Figure 5 shows containment arrows between real input classes. The named inheritance relationship is explicit in the lower box; attributes and methods are detailed in Section 2.3.

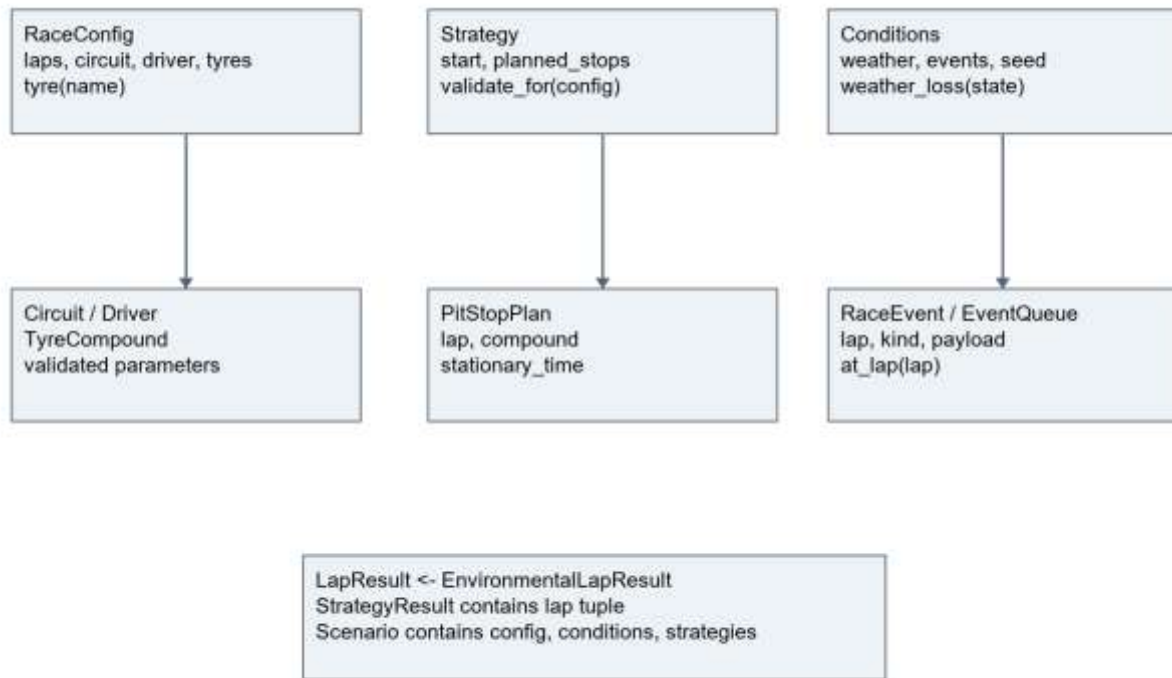


Figure 5: Core class relationships

Complete physics.py as inspected for this report

```

"""Pure explainable formulas; all penalties are measured in seconds."""
from .models import RaceConfig, TyreCompound, integer, number

def degradation(tyre: TyreCompound, age: int) -> float:
    integer(age, "Tyre age")
    loss = tyre.degradation_rate * age
    number(loss, "Degradation loss")
    return loss

def fuel_effect(config: RaceConfig, lap: int) -> float:
    integer(lap, "Lap number", 1, config.laps)
    if config.laps == 1:
        return 0.0
    return config.initial_fuel_penalty * ((config.laps - lap) / (config.laps - 1))

def lap_time(config: RaceConfig, tyre: TyreCompound, age: int,
             lap: int, pit_loss: float = 0.0) -> tuple[float, float, float]:
    number(pit_loss, "Pit loss")
    wear = degradation(tyre, age)
    fuel = fuel_effect(config, lap)
    total = (config.circuit.base_lap_time + config.driver.pace_delta
             + tyre.base_pace_delta + wear + fuel + pit_loss)
    number(total, "Calculated lap time", positive=True)
    return total, wear, fuel
  
```

Appendix B Automated test inventory

Table 32: Final collected cases

Number	Test case	Outcome
001	tests.test_advanced_gui::test_events_parser	PASS
002	tests.test_advanced_gui::test_gui_scenario_roundtrip	PASS
003	tests.test_advanced_gui::test_async_search_and_mc	PASS
004	tests.test_advanced_gui::test_stale_worker_result_discarded	PASS
005	tests.test_advanced_gui::test_worker_cancellation	PASS
006	tests.test_comparison::test_manual_comparison	PASS
007	tests.test_comparison::test_tie_order	PASS
008	tests.test_comparison::test_invalid_comparison	PASS
009	tests.test_conditions::test_weather_transitions	PASS
010	tests.test_conditions::test_weather_ties	PASS
011	tests.test_conditions::test_safety_car_manual	PASS
012	tests.test_conditions::test_invalid_safety_factor[-1]	PASS
013	tests.test_conditions::test_invalid_safety_factor[1.01]	PASS
014	tests.test_conditions::test_invalid_safety_factor[nan]	PASS
015	tests.test_engine::test_five_lap_manual_oracle	PASS
016	tests.test_engine::test_no_stop_manual	PASS
017	tests.test_engine::test_race_boundaries[1]	PASS
018	tests.test_engine::test_race_boundaries[500]	PASS
019	tests.test_engine::test_consecutive_same_compound_stops	PASS
020	tests.test_engine::test_final_stop_rejected	PASS
021	tests.test_engine::test_zero_pit_loss	PASS
022	tests.test_engine::test_overflow_rejected	PASS
023	tests.test_events::test_order_and_ties	PASS
024	tests.test_events::test_bad_events	PASS
025	tests.test_final_robustness::test_json_collection_types[events]	PASS
026	tests.test_final_robustness::test_json_collection_types[planned_stops]	PASS
027	tests.test_final_robustness::test_independent_search_oracle	PASS
028	tests.test_final_robustness::test_trial_reconstruction	PASS
029	tests.test_final_robustness::test_maximum_comparison	PASS
030	tests.test_gui::test_parser	PASS
031	tests.test_gui::test_real_gui_comparison_and_invalidatio	PASS

Number	Test case	Outcome
032	tests.test_gui::test_invalid_stop_error_and_recovery	PASS
033	tests.test_gui::test_single_and_add_remove	PASS
034	tests.test_models::test_race_boundaries[1]	PASS
035	tests.test_models::test_race_boundaries[500]	PASS
036	tests.test_models::test_invalid_race_length[0]	PASS
037	tests.test_models::test_invalid_race_length[-1]	PASS
038	tests.test_models::test_invalid_race_length[501]	PASS
039	tests.test_models::test_invalid_race_length[2.5]	PASS
040	tests.test_models::test_invalid_race_length[True]	PASS
041	tests.test_models::test_invalid_race_length[5]	PASS
042	tests.test_models::test_invalid_parameters[-1]	PASS
043	tests.test_models::test_invalid_parameters[nan]	PASS
044	tests.test_models::test_invalid_parameters[inf]	PASS
045	tests.test_models::test_invalid_parameters[True]	PASS
046	tests.test_models::test_invalid_parameters[fast]	PASS
047	tests.test_models::test_missing_and_duplicate_tyres	PASS
048	tests.test_models::test_invalid_start[None]	PASS
049	tests.test_models::test_invalid_start[SOFT]	PASS
050	tests.test_models::test_invalid_start[0]	PASS
051	tests.test_models::test_stop_order_and_race_end	PASS
052	tests.test_models::test_zero_and_negative_inputs	PASS
053	tests.test_models::test_results_and_state_validation	PASS
054	tests.test_monte_carlo::test_manual_statistics	PASS
055	tests.test_monte_carlo::test_deterministic_and_replay	PASS
056	tests.test_monte_carlo::test_invalid_counts_and_cancel	PASS
057	tests.test_optimiser::test_tiny_space	PASS
058	tests.test_optimiser::test_minimum_stints_and_baselines	PASS
059	tests.test_optimiser::test_limit_and_cancel	PASS
060	tests.test_persistence::test_roundtrip	PASS
061	tests.test_persistence::test_malformed[{}]	PASS
062	tests.test_persistence::test_malformed[{}]	PASS
063	tests.test_persistence::test_malformed[{"x":1,"x":2}]	PASS
064	tests.test_persistence::test_malformed[NaN]	PASS
065	tests.test_persistence::test_malformed[[]]	PASS

Number	Test case	Outcome
066	tests.test_persistence::test_schema_and_model_validation	PASS
067	tests.test_physics::test_manual_degradation[0-0]	PASS
068	tests.test_physics::test_manual_degradation[1-0.2]	PASS
069	tests.test_physics::test_manual_degradation[3-0.6]	PASS
070	tests.test_physics::test_zero_degradation	PASS
071	tests.test_physics::test_invalid_age[-1]	PASS
072	tests.test_physics::test_invalid_age[0.5]	PASS
073	tests.test_physics::test_invalid_age[True]	PASS
074	tests.test_physics::test_fuel_manual	PASS
075	tests.test_physics::test_invalid_lap[0]	PASS
076	tests.test_physics::test_invalid_lap[6]	PASS
077	tests.test_physics::test_invalid_lap[1.5]	PASS
078	tests.test_physics::test_invalid_lap[True]	PASS
079	tests.test_physics::test_lap_manual	PASS
080	tests.test_randomness::test_seed_reproduction	PASS
081	tests.test_randomness::test_bad_seed[-1]	PASS
082	tests.test_randomness::test_bad_seed[4294967296]	PASS
083	tests.test_randomness::test_bad_seed[True]	PASS
084	tests.test_randomness::test_bad_seed[1.5]	PASS
085	tests.test_randomness::test_bad_seed[42]	PASS
086	tests.test_randomness::test_bad_amplitude	PASS
087	tests.test_sc05_tolerance::test_five_lap_oracle_absolute_tolerance	PASS

Appendix C Execution and failure evidence

The full final output is in evidence/test-runs/TEST-QUALITY-FINAL-20260908T224148466396Z/output.txt. The XML records 87 cases without failure/error nodes. Each run's metadata retains command, timestamp and source hashes. Section 3.17 lists all preserved runs; the following final summary is verbatim.

Final pytest output tail with local paths abbreviated

```
tests/test_randomness.py::test_bad_amplitude PASSED [ 98%]
tests/test_sc05_tolerance.py::test_five_lap_oracle_absolute_tolerance PASSED [100%]

- generated xml file: [PROJECT]\evidence\test-runs\TEST-QUALITY-
FINAL-20260908T224148466396Z\results.xml -
===== 87 passed in 8.60s =====
```

Failure source: evidence/test-runs/TEST-I17-ROBUSTNESS-20260908T212523289799Z/output.txt. Complete traceback and source hashes remain untouched. Capture-harness diagnosis: docs/testing/FAIL-I17-02.md; its limitation is a transcript record rather than a separate raw terminal file.

Appendix D Full benchmark values

Verbatim benchmark JSON values

```
{
  "started_utc": "2026-09-08T21:26:52.003607+00:00",
  "machine": {
    "cpu": "Snapdragon(R) X 10-core X1P64100 @ 3.40 GHz",
    "ram_bytes": 16756006912,
    "platform": "Windows-11-10.0.26200-SP0",
    "python": "3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)]",
    "process_architecture": "AMD64"
  },
  "race_laps": 50,
  "repeats": 3,
  "warmup": "One unmeasured single-race run",
  "measurements": [
    {
      "label": "sequential races",
      "count": 1,
      "seconds": [
        0.0005320999989635311,
        0.0005244999993010424,
        0.00051059999896097
      ],
      "minimum": 0.00051059999896097,
      "maximum": 0.0005320999989635311,
      "mean": 0.0005223999990751812,
      "median": 0.0005244999993010424
    },
    {
      "label": "sequential races",
      "count": 100,
      "seconds": [
        0.06135329999960959,
        0.07580089999828488,
        0.06523269999888726
      ],
      "minimum": 0.06135329999960959,
      "maximum": 0.07580089999828488,
      "mean": 0.06746229999892724,
      "median": 0.06523269999888726
    },
    {
      "label": "sequential races",
      "count": 500,
      "seconds": [
        0.34104739999747835,
        0.31911050000053365,
        0.3346101999995881
      ],
      "minimum": 0.31911050000053365,
      "maximum": 0.34104739999747835,
      "mean": 0.3315893666658667,
      "median": 0.3346101999995881
    },
    {
      "label": "sequential races",
      "count": 1000,
      "seconds": [
        0.807915099998354,
        0.6702942000010808,
        0.9107024000004458
      ],
      "minimum": 0.6702942000010808,
      "maximum": 0.9107024000004458,
      "mean": 0.7963038999999602,
      "median": 0.807915099998354
    },
    {
      "label": "candidate prefix scoring (not full search)",
      "count": 10,
      "seconds": [
        0.006008200001815567,
        0.009975800003303448,
        0.009136799999396317
      ],
      "minimum": 0.006008200001815567,
```

```

    "maximum": 0.009975800003303448,
    "mean": 0.00837360000150511,
    "median": 0.009136799999396317
  },
  {
    "label": "candidate prefix scoring (not full search)",
    "count": 100,
    "seconds": [
      0.08367839999846183,
      0.08603339999899617,
      0.08780420000039157
    ],
    "minimum": 0.08367839999846183,
    "maximum": 0.08780420000039157,
    "mean": 0.08583866666594986,
    "median": 0.08603339999899617
  },
  {
    "label": "candidate prefix scoring (not full search)",
    "count": 1000,
    "seconds": [
      0.8923073000005388,
      1.0548605999974825,
      1.0325487000009161
    ],
    "minimum": 0.8923073000005388,
    "maximum": 1.0548605999974825,
    "mean": 0.9932388666663124,
    "median": 1.0325487000009161
  },
  {
    "label": "complete default search",
    "count": 1056,
    "seconds": [
      1.350378200000705,
      1.0296382000015,
      1.1048835000001418
    ],
    "minimum": 1.0296382000015,
    "maximum": 1.350378200000705,
    "mean": 1.1616333000007824,
    "median": 1.1048835000001418
  },
  {
    "label": "paired Monte Carlo trials",
    "count": 100,
    "seconds": [
      0.2067151000010199,
      0.226102399999452,
      0.21768179999708082
    ],
    "minimum": 0.2067151000010199,
    "maximum": 0.226102399999452,
    "mean": 0.21683309999934863,
    "median": 0.21768179999708082
  },
  {
    "label": "paired Monte Carlo trials",
    "count": 500,
    "seconds": [
      1.1139492000002065,
      1.1364091000032204,
      1.2292094000004
    ],
    "minimum": 1.1139492000002065,
    "maximum": 1.2292094000004,
    "mean": 1.1598559000012756,
    "median": 1.1364091000032204
  },
  {
    "label": "paired Monte Carlo trials",
    "count": 1000,
    "seconds": [
      2.516606799999863,
      1.6913468999991892,
      2.089219100002083
    ],
    "minimum": 1.6913468999991892,
    "maximum": 2.516606799999863,

```



```

    "mean": 2.0990576000003784,
    "median": 2.089219100002083
  }
]
}

```

Appendix E Research materials

Section 1.5 contains the three comparative reviews and their ADOPT, ADAPT, REJECT and LEARN decisions. The original record remains docs/analysis/research.md. These sources led to the following review exercises after implementation.

These tasks were added after the source review in docs/analysis/research.md. They are planning material, not feedback or successful tests.

1. Explain why a weather event halfway through lap 10 can only be approximated by the current lap-boundary model. Trace the event heap and resulting penalty.
2. Compare one stop with two stops. Change only pit-lane loss and explain when the ranking changes. Identify the role of missing traffic.
3. Run paired Monte Carlo with SC probability 0,0.3,1 using the same seed and variation. Explain why shared additive noise alone cannot alter the gap, and why SC timing may.
4. Discuss what must be removed or accounted for before using observed lap data to calibrate degradation. Do not paste real laps into configuration and call the model validated.
5. Apply wet weather. Explain why every supported compound receives the same extra penalty and why the application cannot recommend a real wet-tyre strategy.
6. Manually enumerate the 12 legal zero/one-stop plans for a two-lap race and explain the optimiser's grid and tie order.

Record candidate explanations separately and accurately. [GENUINE STAKEHOLDER FEEDBACK PENDING]

Appendix F Human evidence packs

All eight forms below are ready to use but uncompleted. Completed consent and identifying information belong in private-evidence/, not a public repository. Record actual dates and participant aliases. A post-prototype response cannot be presented as an initial interview.

Stakeholder interview

Status: HUMAN INPUT REQUIRED. This form has not been completed.

Participant alias: [REQUIRED]

Session date: [REQUIRED]

Role/experience: [REQUIRED]

Consent record held privately: [REQUIRED]

Use an alias in project records. Keep identifying details and consent records in private-evidence/, outside the public release. Do not backdate a session or describe a post-prototype interview as original research.

Stage: [INITIAL OR POST-PROTOTYPE, RECORD ACTUAL STAGE]

1. What race-strategy decisions would you want to explore?

[GENUINE RESPONSE REQUIRED]

Response space: _____

2. Which race and tyre inputs make sense to you?

[GENUINE RESPONSE REQUIRED]

Response space: _____

3. Which outputs would be most useful, and why?

[GENUINE RESPONSE REQUIRED]

Response space: _____

4. What do you think the stop syntax 15:Medium means?

[GENUINE RESPONSE REQUIRED]

Response space: _____

5. What is confusing or missing?

[GENUINE RESPONSE REQUIRED]

Response space: _____

6. Which simplification could make the results misleading?

[GENUINE RESPONSE REQUIRED]

Response space: _____

7. Which improvement matters most, and why?

[GENUINE RESPONSE REQUIRED]

Response space: _____

Requirements follow-up

Status: HUMAN INPUT REQUIRED. This form has not been completed.

Participant alias: [REQUIRED]

Session date: [REQUIRED]

Role/experience: [REQUIRED]

Consent record held privately: [REQUIRED]

Use an alias in project records. Keep identifying details and consent records in private-evidence/, outside the public release. Do not backdate a session or describe a post-prototype interview as original research.

Table 33: Blank human evidence form

Response reference	Need in participant words	Existing FR / SC	Proposed change and reason	Priority from participant	Acceptance condition	Confirmed date
[REQUIRED]	[REQUIRED]	[REQUIRED]	[REQUIRED]	[REQUIRED]	[REQUIRED]	[REQUIRED]

Ask the participant to confirm this interpretation. Keep disagreement or rejected suggestions and explain the decision. Do not mark requirements agreed until agreement is recorded.

Post-prototype review

Status: HUMAN INPUT REQUIRED. This form has not been completed.

Participant alias: [REQUIRED]

Session date: [REQUIRED]

Role/experience: [REQUIRED]

Consent record held privately: [REQUIRED]

Use an alias in project records. Keep identifying details and consent records in private-evidence/, outside the public release. Do not backdate a session or describe a post-prototype interview as original research.

Build/version shown: [REQUIRED]

Scenario used: [REQUIRED]

1. What did the prototype help you understand?

[GENUINE RESPONSE REQUIRED]

2. Which result or control was difficult to interpret?

[GENUINE RESPONSE REQUIRED]

3. Which single improvement should be prioritised? Why?

[GENUINE RESPONSE REQUIRED]

4. After an implemented change, did it solve the reported difficulty? Record the actual follow-up date and version.

[GENUINE RESPONSE REQUIRED]

Usability task sheet

Status: PLANNED. No completed session is claimed. Use participant aliases and record build, machine, display scale and date on the observation form. Explain that PITWALL is an illustrative simulator. Let the user attempt each task; record any hints rather than silently coaching.

1. Launch PITWALL.

2. Configure a short five-lap race.

3. Create two named strategies.

4. Add a valid stop after lap 2.

5. Compare both strategies.

6. Identify the faster plan and its time advantage.

7. Explain the pit spike on the graph.

8. Enter a stop after the final lap and interpret the error.

9. Add an event and explain when it takes effect.

10. Run a 100-trial Monte Carlo experiment.

11. Explain what the win percentage means.

12. Start optimisation and cancel it; describe the feedback and whether further input is possible.

13. Save a scenario.

14. Change a setting, then reload the scenario and check the restored inputs.

For task 12, if work finishes before cancellation is attempted, record that fact. Do not label it a successful cancellation observation. Use a larger legal search for another attempt and record the changed settings. Timing is optional and must be measured live, never estimated afterwards.

Usability observations

Status: HUMAN INPUT REQUIRED. This form has not been completed.

Participant alias: [REQUIRED]

Session date: [REQUIRED]

Role/experience: [REQUIRED]

Consent record held privately: [REQUIRED]

Use an alias in project records. Keep identifying details and consent records in private-evidence/, outside the public release. Do not backdate a session or describe a post-prototype interview as original research.

Build: [REQUIRED]

Device / OS / display scale: [REQUIRED]

Scenario / settings: [REQUIRED]

Observer alias: [REQUIRED]

Table 34: Blank human evidence form

Task	Completion	Mistakes	Confusion	Help required	Measured seconds or Not measured	Verbatim comment
1	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]
2	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]
3	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]
4	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]
5	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]
6	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]
7	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]
8	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]
9	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]
10	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]
11	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]
12	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]
13	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]
14	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]	[PENDING]

SC04 error meaning in participant words: [PENDING]

SC14 response/cancellation observations, with task settings: [PENDING]

Usability summary

Status: HUMAN INPUT REQUIRED. Do not summarise unused forms as results.

Completed session references: [REQUIRED]

Number of genuine participants: [REQUIRED]

Table 35: Blank human evidence form

Finding and observed task	Evidence / quotation reference	Effect on SC04 / SC08 / SC14	Proposed change	Retest result
[REQUIRED]	[REQUIRED]	[REQUIRED]	[REQUIRED]	[PENDING]

Describe disagreements, assistance provided and session limitations. Distinguish an observed problem from a proposed improvement. Update evaluation only after the evidence exists.

Candidate iteration reflection

Status: CANDIDATE INPUT REQUIRED. Complete a copy for each genuine recorded stage; keep the real review date. A present-day explanation is not a historical diary entry.

Candidate: Mithil Katkoria

Iteration / files: [REQUIRED]

Date personally reviewed: [REQUIRED]

What did you personally review, change or test?

[CANDIDATE FACTUAL INPUT REQUIRED]

Explain the code choice in your own words.

[CANDIDATE FACTUAL INPUT REQUIRED]

Which recorded test did you inspect or actually run? Give its evidence ID.

[CANDIDATE FACTUAL INPUT REQUIRED]

What worked well, based on that evidence?

[CANDIDATE FACTUAL INPUT REQUIRED]

What was difficult for you personally? If nothing is recalled, say so.

[CANDIDATE FACTUAL INPUT REQUIRED]

Which alternative can you explain and why would you choose it?

[CANDIDATE FACTUAL INPUT REQUIRED]

What limitation or uncertainty remains?

[CANDIDATE FACTUAL INPUT REQUIRED]

Which stakeholder response, if any, affected this stage?

[CANDIDATE FACTUAL INPUT REQUIRED]

Which assistance was used and what can you now explain independently?

[CANDIDATE FACTUAL INPUT REQUIRED]

The algorithm-understanding checklist is docs/candidate/algorithm-understanding-check.md. Every topic below is INCOMPLETE. For each, Mithil must record a genuine explanation of purpose, inputs, outputs, algorithm, data structure, edge case, alternative and limitation, with the actual review date.

Table 36: Candidate understanding topics

Topic	Current source	Status
Tyre degradation	physics.py	INCOMPLETE
Fuel model	physics.py	INCOMPLETE
Pit-stop semantics	engine.py	INCOMPLETE
Race loop	engine.py	INCOMPLETE
Event heap	events.py	INCOMPLETE
Weather state	conditions.py / engine.py	INCOMPLETE

Topic	Current source	Status
Safety-car model	conditions.py / engine.py	INCOMPLETE
Random seed	engine.py / conditions.py	INCOMPLETE
Monte Carlo	monte_carlo.py	INCOMPLETE
Statistics	monte_carlo.py	INCOMPLETE
Bounded generation	optimiser.py	INCOMPLETE
Optimisation	optimiser.py	INCOMPLETE
Persistence	persistence.py	INCOMPLETE
GUI worker / cancellation	advanced_gui.py	INCOMPLETE
Validation	models.py / persistence.py / gui.py	INCOMPLETE

No new feature is selected now. A genuine future iteration must cite the feedback, update a requirement, record design and expected tests before implementation, retain actual execution and failures, then obtain a real follow-up review.

Appendix G Git development summary

Table 37: Genuine commits in chronological order

Commit	Recorded timestamp	Subject
51e8e03	2026-09-08T21:58:03+01:00	Design and implement validated core models
1fabfe3	2026-09-08T22:00:49+01:00	Test tyre degradation and deterministic lap formulas
eafc37e	2026-09-08T22:03:22+01:00	Verify dry-race engine and strategy comparison
d6a542c	2026-09-08T22:09:35+01:00	Complete and verify first deterministic desktop milestone
19608f1	2026-09-08T22:16:39+01:00	Add tested events seeded simulation Monte Carlo and bounded search
cdd4bf2	2026-09-08T22:26:34+01:00	Preserve integrated GUI and genuine JSON robustness failures
ecf3829	2026-09-08T22:38:19+01:00	Fix JSON validation and preserve verified prototype evidence pack

These commits are evidence of actual version history, not proof that each design file preceded every edit. The iteration records and tool conversation provide finer-grained sequence. Documentation cleanup changes present at report preparation are uncommitted and are not misrepresented as an extra commit.

Appendix H Evidence catalogue and completeness audit

The pre-report evidence index contains 214 entries. Every listed file and SHA256 hash was checked before report generation. The index includes code, planning records and clearly labelled placeholders as well as captured output; 214 entries does not mean that many screenshots or completed human studies. report/evidence-index.csv remains the full catalogue.

Table 38: Criterion completeness and remaining work

Criterion	Section	Evidence used	Status	Remaining work
3.1.1(a) Computational problem	1.1-1.3	Requirements and model explanation	STRONG	Explain repeated scenario evaluation and parameter trade-offs

Criterion	Section	Evidence used	Status	Remaining work
3.1.1(b) Computational approach	1.3	Decomposition, state and iteration	STRONG	Complete understanding check
3.1.2(a) Stakeholders	1.4; 5.8	Relevant groups and blank interviews	HUMAN INPUT REQUIRED	Conduct interview and requirements follow-up
3.1.3(a) Research	1.5	Three cited post-prototype reviews	PARTIAL	Candidate review; use future findings prospectively
3.1.3(b) Essential features	1.7	FR01-FR12 with rationale	STRONG	Confirm requirements with stakeholder
3.1.3(c) Limitations	1.8; 5.10	Model and deployment limits	STRONG	Discuss real user consequences
3.1.4(a) Requirements	1.9	Runtime, hardware and benchmark environment	STRONG	Validate on intended user machine
3.1.4(b) Success criteria	1.10; 5.2	SC01-SC14 and explicit evidence	STRONG	Confirm success criteria with stakeholder
3.2.1(a) Decomposition	2.1	Problem and component breakdown	STRONG	Explain boundaries and state
3.2.2(a) Structure	2.2-2.3	Current-source architecture and models	PARTIAL	Preserve labels; prospective design only after feedback
3.2.2(b) Algorithms	2.6	A01-A14 and source extracts	STRONG	Complete all 15 understanding topics
3.2.2(c) Usability design	2.8-2.9	GUI captures, labels, validation and worker design	PARTIAL	Run task sheet and act on actual findings
3.2.2(d) Data and validation	2.3-2.5; 2.7	Typed models, variables, strict schema	STRONG	Explain edge cases and alternatives
3.2.3(a) Test data	2.10-2.11; 4	Recorded stage plans and final quality plan	STRONG	Complete planned human tasks
3.3.1(a) Iterative process	3.1-3.15; App G	Recorded stages, source, seven commits	CANDIDATE INPUT REQUIRED	Complete dated factual reflections
3.3.1(b) Prototypes	3.6; 3.14	First and advanced genuine GUI captures	STRONG	Explain preserved versions; do not fabricate missing captures
3.3.2(a) Development tests	3.17; App B	17 original runs plus two new runs	STRONG	Explain meaningful expected results
3.3.2(b) Remedial action	3.16	I17 JSON failure, remedy, retest and regression	STRONG	Explain empty-container cause and remedy
3.4.1(a) Final robustness	4.1-4.4	87 passing cases and genuine timings	STRONG	Review coverage limits
3.4.1(b) Usability tests	4.5	Blank 14-task protocol	HUMAN INPUT REQUIRED	Run sessions and preserve observations
3.4.2(a) Success evaluation	5.2	Criterion evidence and explicit judgements	PARTIAL	Update after real sessions
3.4.3(a) Usability effectiveness	5.7-5.8	Real captures and automated GUI tests	HUMAN INPUT REQUIRED	Evaluate observed task outcomes
3.4.4(a) Maintenance	5.9	Six concrete module/change concerns	STRONG	Explain consequences and regression needs

Criterion	Section	Evidence used	Status	Remaining work
3.4.4(b) Further development	5.10-5.11	Eight limitations and possible changes	STRONG	Choose one only after actual feedback

Official OCR content points 3.1 through 3.4 and the published marking bands were checked against the specification downloaded on 8 September 2026. The 24-point mapping above uses the official numbering. The separate centre Full Mark Criteria document was not found, so centre-specific integration is TEACHER INPUT REQUIRED. The full strengths, risks, actions and human responsibilities are in report/FINAL_CRITERIA_AUDIT.md. No mark is awarded by this review.

Appendix I JSON contract and example

Schema version1 requires config, conditions and strategies, with exact field sets. Enum values are stored as strings. Arrays are explicitly checked after the genuine fix. There is no standalone formal JSON Schema file; the contract is implemented in persistence.py and validated by model constructors. Search/Monte Carlo controls are stored in experiment exports, not the base scenario.

Actual dry-comparison.json example

```
{
  "schema_version": 1,
  "config": {
    "laps": 50,
    "circuit": {
      "name": "Custom circuit",
      "base_lap_time": 90.0,
      "pit_lane_loss": 20.0
    },
    "driver": {
      "name": "Custom driver",
      "pace_delta": 0.0
    },
    "tyres": [
      {
        "name": "Soft",
        "base_pace_delta": 0.0,
        "degradation_rate": 0.16,
        "recommended_life": 15
      },
      {
        "name": "Medium",
        "base_pace_delta": 0.5,
        "degradation_rate": 0.09,
        "recommended_life": 25
      },
      {
        "name": "Hard",
        "base_pace_delta": 1.0,
        "degradation_rate": 0.045,
        "recommended_life": 35
      }
    ],
    "initial_fuel_penalty": 3.0
  },
  "conditions": {
    "initial_weather": "Dry",
    "events": [],
    "damp_penalty": 4.0,
    "wet_penalty": 15.0,
    "safety_car_penalty": 25.0,
    "safety_car_pit_factor": 0.5,
    "seed": 42,
    "variation": 0.0
  },
  "strategies": [
    {
      "name": "One stop",
      "starting_compound": "Medium",
      "planned_stops": [
        {
```



```
        "lap": 25,
        "compound": "Hard",
        "stationary_time": 2.5
    }
},
{
    "name": "Two stops",
    "starting_compound": "Soft",
    "planned_stops": [
        {
            "lap": 15,
            "compound": "Medium",
            "stationary_time": 2.5
        },
        {
            "lap": 32,
            "compound": "Hard",
            "stationary_time": 2.5
        }
    ]
}
]
```

Appendix J Additional application screenshots

Figure 6 shows totals, stop count, average, fastest and slowest laps, pit loss and stint lengths.

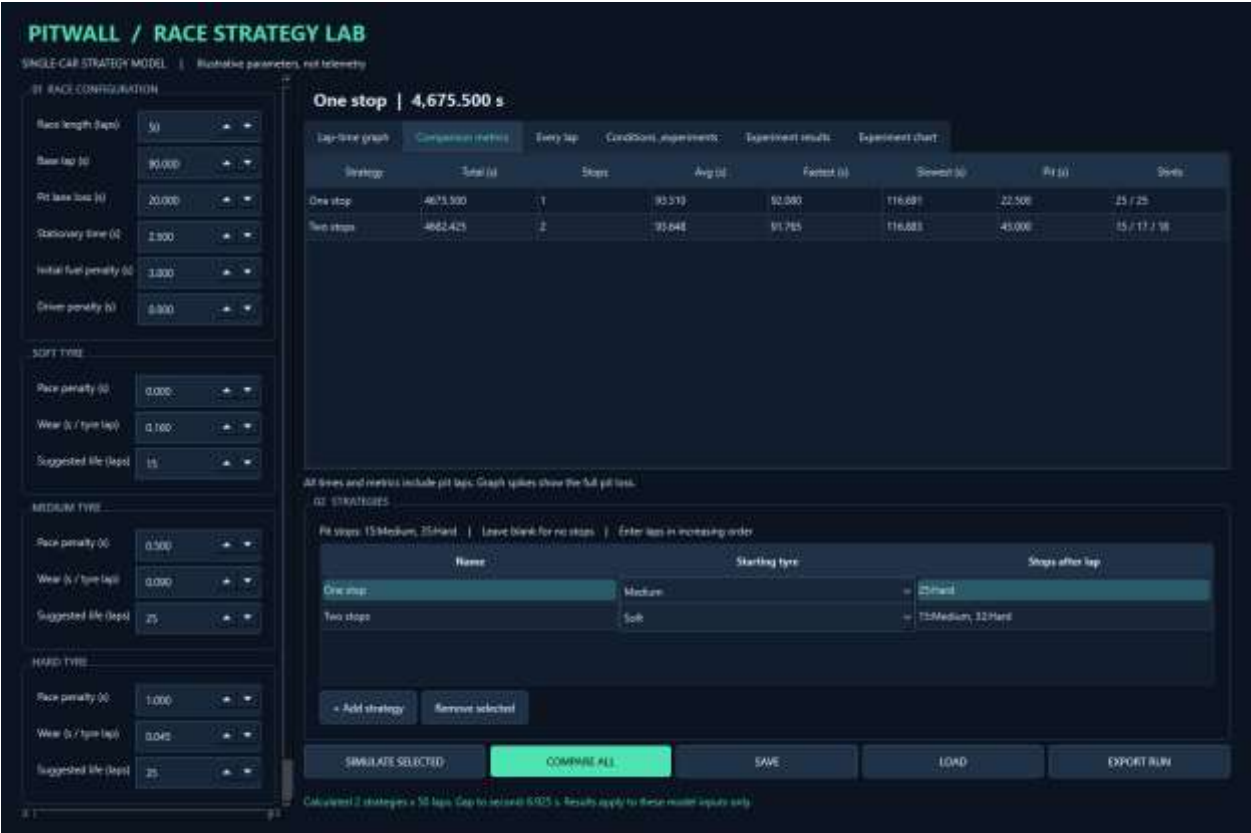


Figure 6: Actual comparison metrics

Figure 7 shows the per-lap model components for inspection.

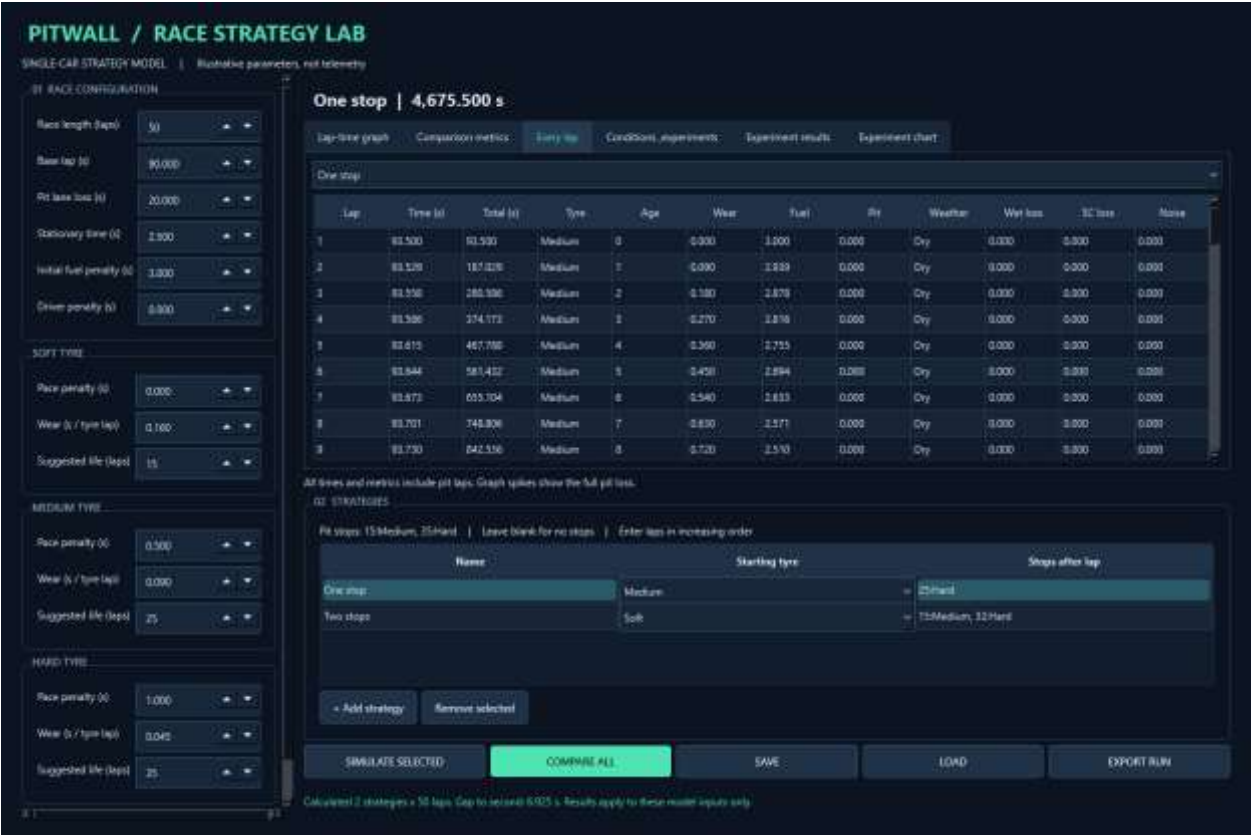


Figure 7: Actual lap records

Figure 8 shows the scrollable environment controls; controls below the visible region require scrolling.

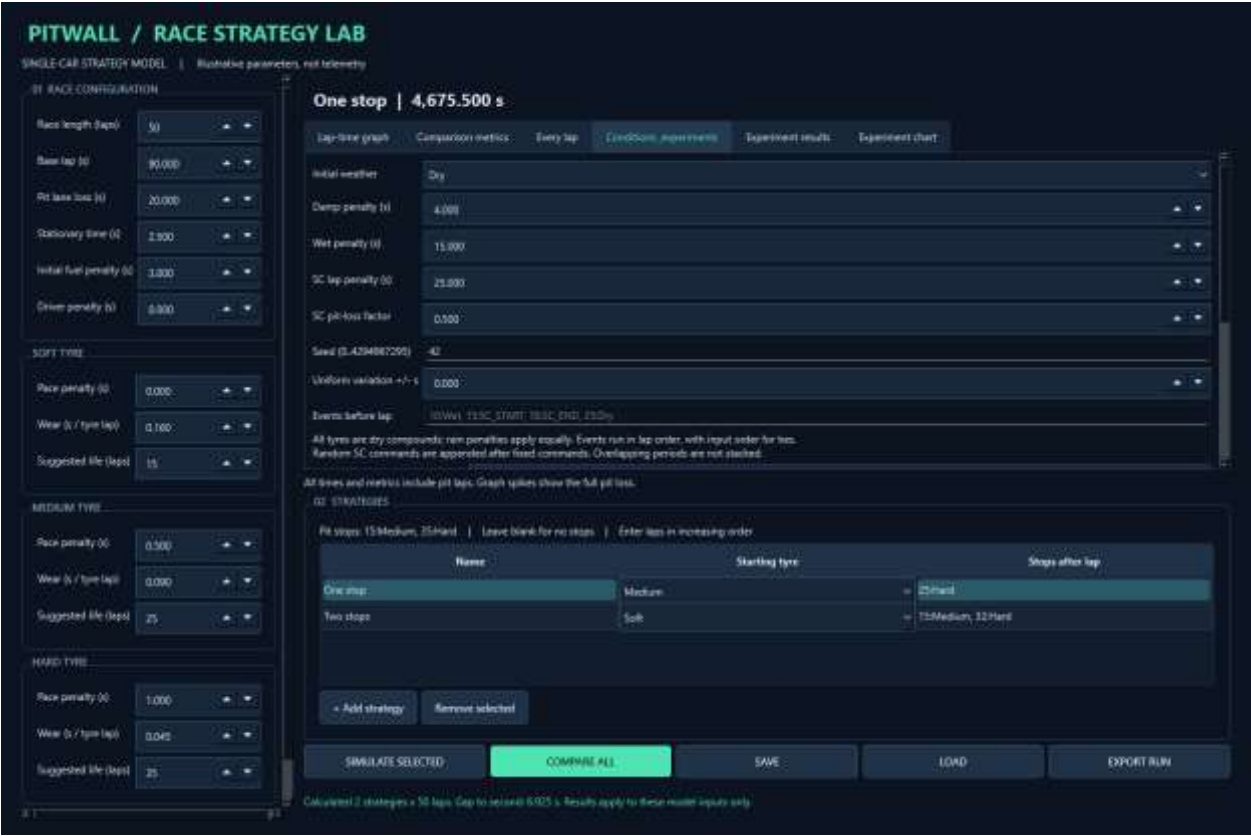


Figure 8: Conditions and experiment settings

Figure 9 shows 100 recorded trials; A wins88% under the chosen illustrative settings.

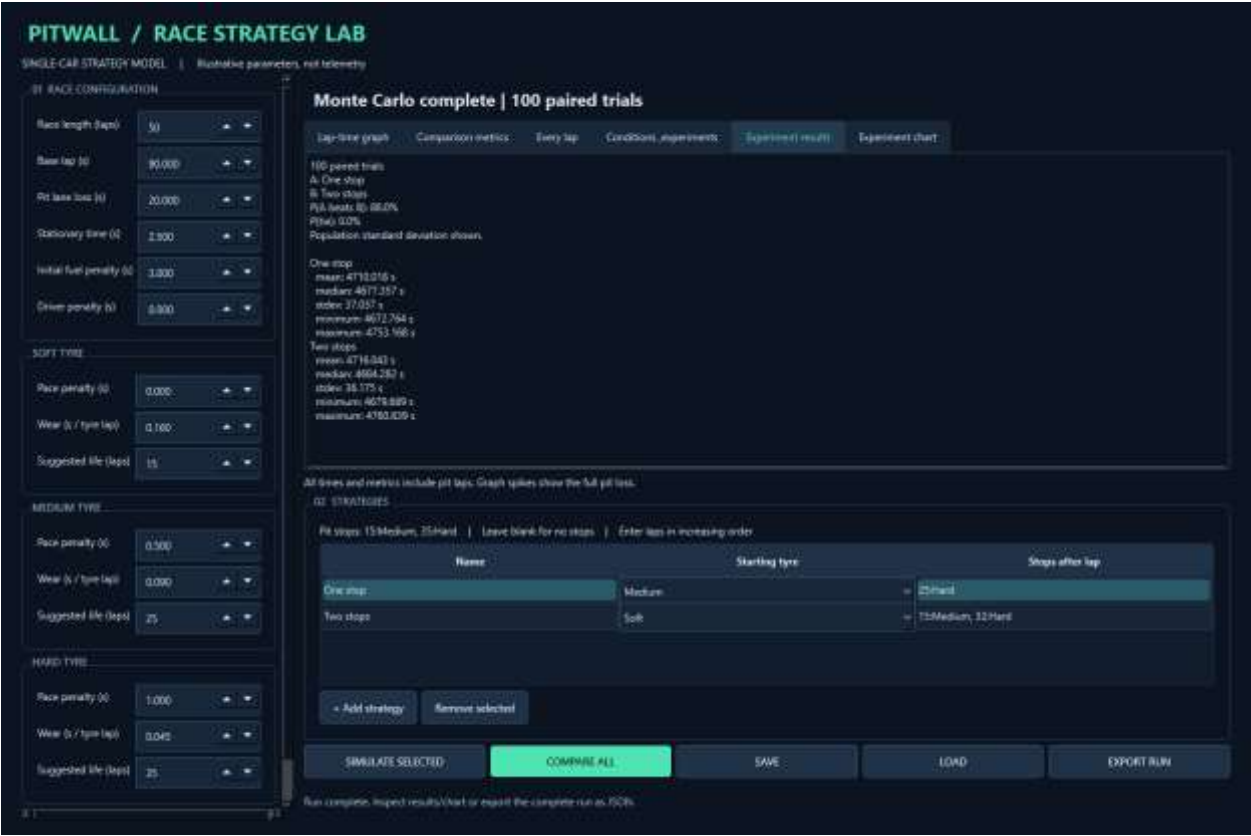


Figure 9: Actual paired Monte Carlo summary

Figure 10 plots the same recorded trial totals with shared bins.



Figure 10: Actual Monte Carlo histogram

Appendix K Development declaration

Tool used: Codex. Centre-specific declaration guidance is pending. No assistance percentage is assigned.

Table 39: Factual assistance record

Date / stage	Assistance provided	Affected files / features	Candidate follow-up recorded
2026-09-08, planning and records	Drafted requirements, designs, diagrams, development records, README and evaluation; reviewed three sources after the prototype	docs/, README.md, report/	Candidate supplied brief; subsequent review not recorded
2026-09-08, implementation	Authored simulation, comparison, GUI, environmental events, randomness, Monte Carlo, search and persistence modules	src/pitwall/, main.py, launch.ps1	Personal changes or explanation not recorded
2026-09-08, verification	Wrote/ran tests; fixed JSON validation and capture-harness issues; captured/inspected actual GUI output and ran benchmarks	tests/, scripts/, evidence/	Personal testing or review not recorded
2026-09-08, cleanup	Reorganised public documentation and consolidated attribution	Files in report/documentation-cleanup.md	Candidate requested cleanup; review of edits not recorded
2026-09-08, master report	Assembled and checked the evidence-based DOCX/PDF, diagrams and completeness audit	report/; SC05 evidence judgement in docs/evaluation/review.md	Candidate requested report; personal review and missing human evidence remain pending
2026-09-08, final quality pass	Added explicit tolerance verification and comparison benchmark; prepared blank human forms, final report and local release files	tests/test_sc05_tolerance.py, scripts/benchmark_comparison.py, docs/, report/, public packaging	Candidate requested quality pass; personal explanations, human sessions and centre approval remain pending

Source review involved no hands-on trials or copied source code/parameters. Execution records, captures and Git history remain intact. Add personal review, changes, tests and explanations when completed.

The record is maintained separately at docs/assistance-log.md. Candidate follow-up must be completed factually; this report does not replace the centre's declaration process.

The public release is staged locally. Publication is pending centre permission and privacy approval. Local absolute paths in report excerpts are abbreviated as [PROJECT] or [PYTHON]; original execution records remain intact in the private evidence repository.